



OPEN

A direction aware predictive offloading framework for energy conscious mobile augmented reality systems

S. Anitha Jebamani[✉] & Godfrey Winster Sathianesan

Mobile Augmented Reality (MAR) applications require efficient and quick real-time object recognition with minimal latency in order to maintain user immersion. However, these applications are severely affected by energy constraints and dynamic user movement. Conventional offloading models often neglecting direction-based user behavior incur unnecessary processing and communication overhead which further leads to inefficient resource utilization and degraded responsiveness. Methods based on reactive task scheduling fail to predict user motion which leads to delayed execution and increased energy consumption. To overcome these limitations, a novel Directional Awareness-Driven Energy Efficient Offloading (DEEPO) model is presented in this research work. The proposed model integrates a Dual-Gated Recurrent Spatial-Temporal Predictor (DG-RSTP) for user direction prediction and NanoDet-Plus for lightweight, high-speed object detection. Also, it employs Region-Based Task Prefetching and Mapping (RTPM) with Bipartite Energy-Distance Cost Optimization to selectively offload tasks to edge nodes within the user's predicted field of view. Additionally, an energy-efficient communication is incorporated which ensures Adaptive Packet Thinning and Beam-Shift Aware Transmission. Experimental evaluation on real-world mobility and object datasets demonstrates that DEEPO achieves substantial performance gains over state-of-the-art schedulers. The proposed DG-RSTP predictor improves trajectory accuracy by up to 11.2%, while the region-specific prefetching and bipartite optimization jointly reduce offloading latency by 19.4% and energy consumption by 14.7% compared to baseline methods. DEEPO also lowers SLA violation rates by 17.8%, confirming its effectiveness under dynamic user motion. These results establish DEEPO as a robust and predictive offloading framework capable of sustaining energy-efficient and responsive mobile AR experiences.

Keywords Predictive offloading, Mobile augmented reality, Energy efficiency, Deep learning, Edge, Directional object detection

The demand for ultra-responsive and energy-conscious task handling has increased rapidly due to intelligent edge-enabled applications, particularly in vehicular interaction, immersive navigation, and real-time urban surveillance. In such cases efficient decision-making must occur near the user, reducing dependency on centralized data centers and minimizing end-to-end communication latency. As mobile devices interact with high-speed moving subjects and dense data environments, traditional cloud-based offloading models fall short in providing timely computational support. Instead, decentralized systems supported by mobile edge computing nodes offer a viable platform to sustain task execution with lower delays, enhanced contextual awareness, and reduced transmission overheads¹. Within these systems, predictive task assignment where computations are directed proactively rather than reactively as it is essential. In parallel, user behavior patterns, visual scene interpretation, and resource constraints must be cohesively integrated to maintain high service reliability. Thus, there exists an urgent requirement for frameworks that can combine contextual mobility, object-driven task generation, and resource-sensitive offloading mechanisms into a seamless pipeline.

Despite substantial progress in the development of energy-aware cloud management systems, several research gaps persist in current scheduling methodologies. Most existing strategies rely on static or semi-static optimization rules that lack real-time adaptive steering to cope with unpredictable fluctuations in workload

Department of Computing Technologies, School of Computing, College of Engineering & Technology, SRM Institute of Science & Technology, Kattankulathur, Chennai 603203, India. ✉email: as5104@srmist.edu.in

intensity, network congestion, and energy availability. Moreover, many frameworks fail to account for heterogeneous resource characteristics across diverse computing layers, resulting in inefficient task distribution and degraded service quality under mixed workloads. Another critical limitation lies in the absence of integrated federated learning (FL) and reinforcement learning (RL) coordination mechanisms that can enable decentralized, self-evolving scheduling decisions while minimizing communication overhead. These shortcomings collectively restrict scalability, responsiveness, and energy optimization potential in large-scale green cloud infrastructures.

Other than the benefits of utilizing distributed computing multiple challenges occur while implementing the same in real time. The unpredictable changes in user direction and behavior are a serious challenge as it leads to inaccurate task dispatching. This further increases the queue, misallocation of resources, and increased delay penalties. In addition to that, mobile devices need to maintain the energy efficiency and blindly transmitting all sensory data to edge nodes will reduce battery lifetime drastically. Also, it brings redundant processing overhead. Existing scheduling techniques considered the tasks as discrete and context-independent, without considering the motion trajectories or visual intent which further leads to a mismatch between computational supply and demand². The inability to project user paths over time further complicates the load balancing across edge units, affects energy efficiency, and degrades the system responsiveness in high-mobility environments.

Although several studies have explored energy-aware and context-driven task scheduling, most existing frameworks still exhibit three core deficiencies. First, they depend on reactive or threshold-based policies that fail to anticipate user mobility or directional intent, leading to delayed task allocation and redundant computation. Second, object detection and contextual sensing modules in current systems operate independently from the offloading engine, preventing coordinated decision-making between what is observed and where the task should execute. Third, existing multi-objective optimization schemes treat energy and latency as isolated goals without a unified trade-off formulation, limiting adaptability under fluctuating workloads. The proposed DEEPO framework directly addresses these shortcomings by coupling motion forecasting, region-specific task prefetching, and energy–distance bipartite optimization within a single predictive pipeline, thereby improving responsiveness and sustainability in dynamic mobile augmented-reality environments.

Numerous solutions have been explored to reduce the latency and enhance the computational throughput in edge networks. Trajectory-based motion forecasting models have been widely used in various applications^{3,4} however, these models depend on recurrent structures or attention-based networks for extracting the temporal patterns in motion sequences. Though these techniques exhibit moderate accuracy in short-range prediction but those models exhibit error propagation over longer time. Recent edge computing research has focused extensively on binary or heuristic offloading schemes in which static thresholds are used to decide whether a task should be handled locally or remotely⁵. However, these models fail to scale with quick mobility and dynamic computation availability. Additionally, energy-efficient task dispatching models dependent on user orientation and task relevance which makes them unsuitable for applications on field-of-view or forward motion direction.

To address these challenges, this study introduces CloudPulse, a direction-aware, adaptive, and energy-conscious scheduling framework for green cloud environments. Unlike conventional systems that make isolated scheduling decisions, CloudPulse incorporates an integrated FL-RL hybrid coordination layer that continuously learns from distributed node behaviors and dynamically optimizes scheduling based on real-time environmental feedback. This integration enables decentralized decision-making without compromising global performance objectives, ensuring both energy efficiency and scalability across heterogeneous infrastructure layers.

Recently researches have been made to connect visual analysis with computational resource allocation. Object-centric decision policies have emerged in drone-based surveillance and AR/VR applications in which visual entities trigger the computational workflows. However, the coupling of detected elements with their motion based spatial relevance are underutilized^{6,7}. Moreover, current systems rarely employ adaptive prioritization based on object significance, spatial scale, or visual confidence. Furthermore, although several lightweight object detection frameworks have been introduced for embedded environments, their integration into predictive edge scheduling remains rudimentary. These models typically detect and report visual data but do not participate in dynamic task generation or execution strategies⁸. These shortcomings motivate the need for a unified model capable of predicting motion, isolating directional fields of interest, generating prioritized tasks, and mapping them to suitable edge units with minimal energy and delay penalties. The objective is to build a framework that learns the interplay between user intent, object significance, and network state to orchestrate context-aware task offloading.

In conventional edge-based systems, user mobility introduces rapid changes in spatial context. When a user moves unpredictably, the tasks pre-allocated to nearby edge nodes often become irrelevant because the user exits the coverage area before the computation completes, resulting in wasted processing cycles and stale outputs. Similarly, reactive schedulers trigger task offloading only after a change is detected, causing repeated re-sensing, re-transmission, and re-allocation of tasks. These repeated cycles increase communication overhead and lead to unnecessary energy expenditure on both the mobile device and edge servers. Therefore, without predictive modeling of motion and context, traditional systems inherently suffer from resource waste and elevated energy consumption.

This study introduces a structured predictive offloading solution titled DEEPO, designed to integrate direction forecasting, priority-guided task formulation, and energy-efficient edge allocation into a single decision framework. Unlike previous works that handle these operations independently, the proposed method employs a dual-encoding temporal forecasting layer that predicts user trajectory using compressed spatial patterns. This is followed by a dynamic region extraction process that isolates relevant image segments based on directional projections. The visual data is then processed to extract objects of computational interest, from which task vectors are derived based on bounding area, class relevance, and detection strength. These vectors are then prioritized and evaluated through a system-wide objective function that considers task urgency, communication cost, and computational delay. By incorporating a multi-criteria optimization layer, the model dynamically selects the

most suitable destination for each task. The novelty lies in the coupling of movement-aware context modeling with object-sensitive offloading, all while ensuring real-time adaptability in volatile network conditions.

The main contributions of this research are summarized as follows:

1. A detailed analysis of existing energy-efficient scheduling models is conducted, revealing limitations such as lack of adaptive control, poor heterogeneity handling, and minimal integration of learning-based coordination strategies.
2. Introduction of a unified FL-RL-driven scheduling model that enables decentralized, adaptive, and energy-conscious task distribution across diverse nodes.
3. Formulation of a multi-objective optimization mechanism balancing energy efficiency, response time, and load distribution under varying workloads.
4. Comprehensive simulation using benchmark workloads and dynamic network conditions demonstrating CloudPulse's superiority in energy savings, latency reduction, and scheduling stability compared to existing techniques.
5. The proposed framework advances sustainable computing objectives by minimizing energy footprints in large-scale cloud infrastructures, aligning with UN Sustainable Development Goals 7 (Affordable and Clean Energy) and 13 (Climate Action).

The core problem addressed in this work is the lack of a predictive, direction-aware, and energy-optimized offloading mechanism for mobile AR systems operating under continuous motion. Existing approaches are predominantly reactive and treat energy, latency, and mobility as isolated objectives, resulting in suboptimal allocation under dynamic movement. The novelty of DEEPO lies in three aspects: (i) a DG-RSTP predictor that forecasts user direction using a dual-gated GRU-LSTM architecture, (ii) a region-specific prefetching process that prioritizes tasks along the predicted motion path, and (iii) an energy-distance bipartite optimization model that selects the best edge nodes by jointly minimizing energy cost and spatial transmission distance. Together, these components form a unified predictive pipeline that significantly enhances responsiveness and sustainability in real-world mobile AR environments.

The structure of this paper is organized as follows: the second section reviews earlier research relevant to motion tracking and edge-based task handling; the third section details the complete design and computation logic; the fourth section discusses test results; and the final section highlights the key outcomes and future directions.

Related works

Recent advancements in edge computing have led to numerous strategies for task offloading, mobility prediction, and intelligent resource scheduling. This section outlines key contributions that explore direction estimation, object-based computation, and distributed decision-making, highlighting their methodologies, effectiveness, and existing shortcomings in handling spatial context and predictive offloading.

A learning based offloading model presented in⁹ integrates long short-term memory (LSTM) for mobility prediction and Q-learning for task distribution in vehicular edge computing. The framework anticipates vehicle movement patterns and adapts task assignments accordingly, aiming to reduce service delay. Experiments evaluations utilized realistic mobility traces and variable offloading loads to demonstrate the improved latency compared to conventional heuristic schemes. However, the presented model depends on a centralized coordination strategy which limits the model scalability in high-density traffic scenarios.

An edge intelligence-based task offloading scheme is presented in¹⁰ employed federated deep reinforcement learning to handle the service distribution across roadside edge nodes. The presented model integrates a dual-agent strategy in which one agent handles the mobility-aware resource allocation and the other adapts service migration decisions based on edge node load and user trajectory. Experimental analysis which considered urban traffic conditions exhibits the model improvements in execution time and bandwidth usage compared to conventional approaches. However, the presented method requires consistent training synchronization among distributed agents which may not possible in dynamic vehicular settings.

The multi-agent edge offloading system presented in¹¹ utilized deep deterministic policy gradient (DDPG) to improve decision-making in resource-constrained networks. The presented approach decomposes task scheduling into cooperative learning among agents in which each representing a service node. The experimental results demonstrate the improved performance gains in terms of reduced latency and increased task success rate. However, the model depends heavily on continuous reward updates and stable communication among agents, which may not be suitable for high-mobility scenarios.

A joint optimization strategy presented in¹² utilizes a hierarchical deep reinforcement learning (HDRL) framework for offloading and trajectory planning in edge networks. The system decomposes the problem into two coordinated layers in which one optimizing the path based on energy and coverage constraints, and another handles the task placement across edge nodes. Experimental analysis demonstrates the improvements of proposed model in terms of latency reduction and balanced resource compared to standard Q-learning and heuristic approaches. However, the presented model assumes predictable mobility and uninterrupted communication which fails to adapt the complex real-world settings.

The context-aware task offloading model presented in¹³ integrates semantic data classification with deep Q-network-based scheduling in edge computing. The presented model categorizes tasks based on content type and urgency which allows differentiated treatment during offloading. Further a learning agent is used to evaluate the available resources and user state to dynamically assign tasks with minimized delay and system overload. Experimental evaluations exhibited the model enhanced task acceptance rates and latency control when

compared with baseline methods. However, the presented model does not consider the visual field constraints which reduces its effectiveness in applications requiring spatial prediction.

The distributed task management model presented in¹⁴ utilized a multi-agent reinforcement learning model specifically edge interaction. In the presented model each agent operates independently to evaluate local resource states and make task forwarding decisions. This enhances the scalability and responsiveness in dense environments. The presented model also employs policy sharing to improve convergence across agents and ensures robustness to partial observability. Experimental evaluations highlight the model improved performance in processing time, service ratio, and bandwidth efficiency under fluctuating network conditions. However, the presented model lacks in incorporating user trajectory into the decision process which limit its suitability in context-specific offloading scenarios.

The collaborative edge computing strategy presented in¹⁵ incorporates task replication and deadline-aware scheduling. The presented system evaluates the proximity and load of multiple edge servers to replicate computational tasks across suitable nodes which enhances the fault tolerance. The optimization process which is formulated using a constrained utility maximization model demonstrate the improvements in deadline satisfaction and processing reliability. However, the presented model assumes prior knowledge of task criticality and does not integrate user mobility prediction which limits its effectiveness in trajectory-sensitive applications.

The hybrid model presented in¹⁶ utilized graph-based offloading which is combined with bandwidth-aware resource estimation in dynamic edge computing environments. The presented approach models the task allocation problem as a weighted bipartite graph in which edges are used to represent the suitability of node-task pairs based on bandwidth availability and execution urgency. Also, a matching algorithm is applied to maximize the total system utility while balancing load across nodes. Experimental analysis confirmed the model improved offloading efficiency and reduced latency in bandwidth-constrained conditions. However, the presented model does not incorporate motion prediction from the user side which restrict the adaptability in applications requiring foresight in user interaction or directional relevance.

The task offloading model presented in¹⁷ enhanced edge-cloud cooperation by network slicing and dynamic priority adjustment. The presented model categorizes tasks into service slices based on urgency and resource demand with each slice allocated to specific computational domains depending on current network status. The utility function which guides the offloading decisions to balance processing speed and service quality. Simulation analysis demonstrates the improvements in load distribution and task response time across varying traffic loads. However, the presented model lacks in predictive modeling for user movement and does not incorporate directional task filtering. This affects the performance in terms of reduce efficiency in scenarios requiring spatial awareness.

A two-stage decision model presented in¹⁸ for partitioning and offloading convolutional neural network layers in IoT systems incorporated deep reinforcement learning coupled with the Salp Swarm Algorithm. The model separates the offloading decision and resource allocation processes, enabling adaptive optimization based on dynamic environmental conditions. CNN segments are selectively deployed to edge servers to reduce latency and power draw on resource-constrained IoT devices. Simulations were conducted under varying server loads and task complexities, with performance compared to fully local, fully offloaded, and JROPSO strategies. Although effective in reducing execution cost, the method lacks spatial awareness or user-mobility prediction, which limits its use in motion-centric edge scenarios.

In addition to traditional optimization and heuristic approaches, recent years have witnessed a strong shift toward federated and reinforcement learning-based paradigms for cloud and edge resource management. The FedOpt and FedDRL frameworks^{22–24} combine global aggregation with local reinforcement learning agents to dynamically allocate workloads while minimizing energy consumption across data centers. Similarly, MetaFedRL integrates meta-learning with actor-critic reinforcement modules, enabling nodes to generalize across heterogeneous workloads and maintain optimal power–performance trade-offs. Another recent system, HierRLCloud^{25,26}, adopts a hierarchical RL structure where global agents coordinate multiple local controllers to balance energy efficiency and service latency in multi-tier cloud infrastructures. In parallel, policy-gradient-based schedulers such as DDPGCloud and PPO-FedEdge leverage continuous action spaces for adaptive scaling and VM migration under varying thermal and power constraints. These methods collectively demonstrate the rapid advancement of hybrid FL–RL mechanisms, motivating the need for an integrated, direction-aware, and energy-conscious scheduling framework such as CloudPulse.

Recent studies have significantly advanced computation offloading and resource management in mobile and edge computing through learning-based and optimization-driven approaches. A deep reinforcement learning framework using a dueling DQN is introduced in²⁷ to jointly optimize partial computation offloading and resource allocation in mobile edge computing, achieving reduced latency and energy consumption under dynamic network conditions. In a collaborative edge environment, a multi-agent deep deterministic policy gradient (MADDPG) strategy is proposed in²⁸ to simultaneously address task offloading and resource pricing, ensuring improved system utility and fairness among multiple edge agents. Sustainability is emphasized in²⁹, where a multi-objective edge computing model is developed to minimize energy consumption while maintaining performance, demonstrating notable improvements in energy efficiency. Real-time scheduling challenges in dynamic IoT systems are addressed in³⁰ through an efficient CPU/GPU heterogeneous task scheduling mechanism that enhances GPU utilization and memory efficiency under varying workloads. In contrast, the classical optimization-based framework presented in³¹ establishes fundamental models for efficient multi-user computation offloading in mobile-edge cloud computing, serving as a baseline for latency–energy trade-off analysis and influencing the design of more recent learning-based edge computing solutions.

Research gap From the review it is identified that existing methods fall short in integrating user trajectory prediction, directional awareness, and visual-context-based task generation into a unified decision-making

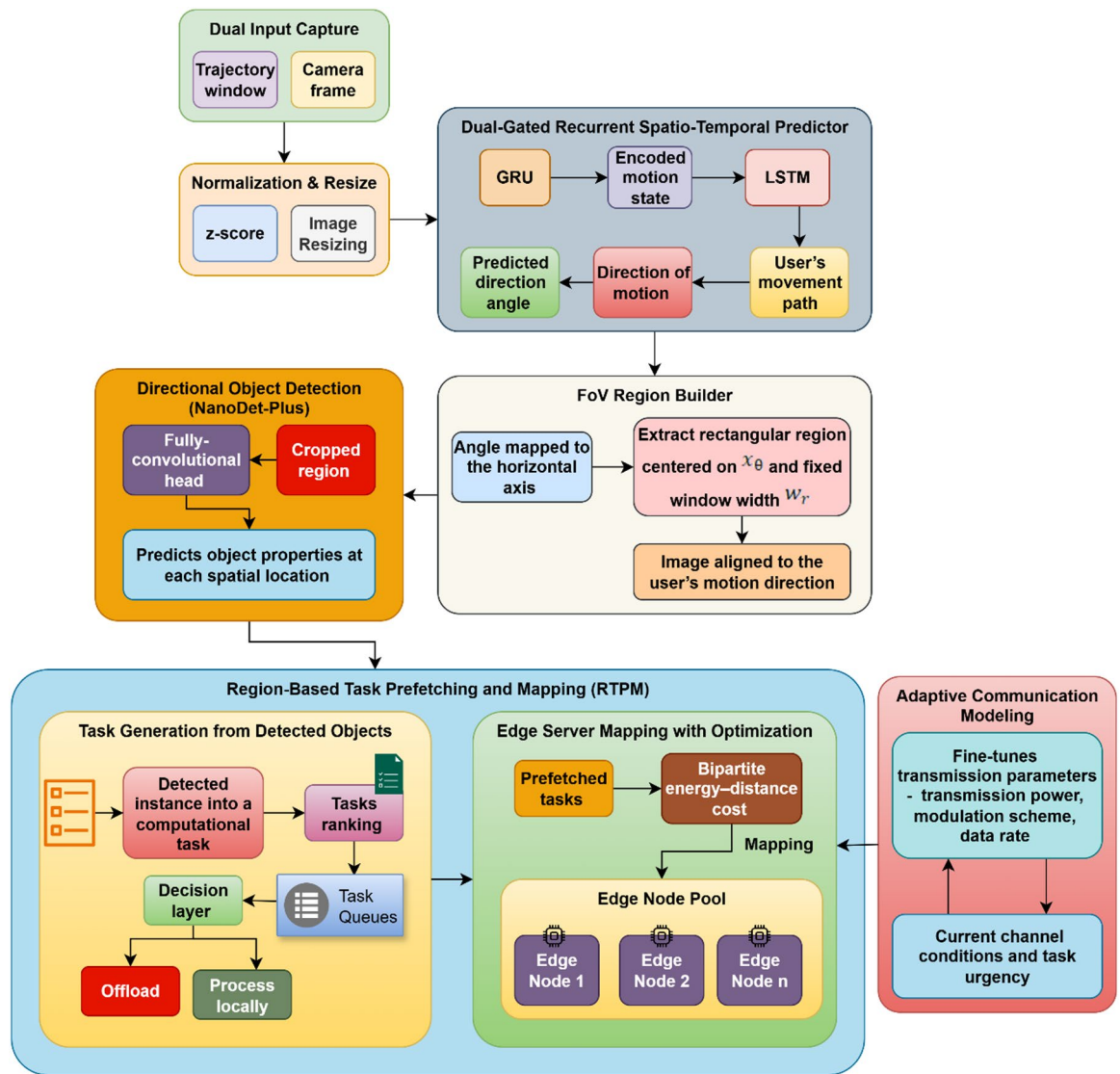


Fig. 1. DEEPO Framework and Process Flow.

pipeline. Many approaches rely on mobility prediction using recurrent models or reinforcement learning for resource distribution, yet they often disregard the spatial relevance of detected objects and their forward alignment with the user's motion. Furthermore, current frameworks typically treat tasks as isolated units without considering the contextual linkage between field-of-view, object priority, and path continuity. Several models emphasize network performance or agent coordination but overlook the importance of adaptive task formulation based on what the user is likely to encounter ahead. Moreover, task replication and scheduling algorithms are generally designed without coupling computational urgency to visual cues, resulting in redundant processing or delayed execution. These limitations indicate a lack of holistic systems that can dynamically capture directional cues, extract relevant visual information, translate that into computational tasks, and map them efficiently across edge nodes with consideration for energy, delay, and resource constraints. This gap underscores the necessity for a predictive, direction-guided, object-aware offloading framework such as the one proposed in the current study.

In summary, while prior models have explored aspects of resource scheduling, energy optimization, and mobility prediction, most fail to achieve a unified understanding of spatio-visual dynamics. The absence of direction-aware modeling results in redundant task assignments that disregard user movement continuity, while the lack of visual-mobility integration prevents contextual task prioritization based on environmental relevance. Moreover, existing frameworks often decouple perception from computation, causing inefficiencies in both latency and energy utilization. These shortcomings underscore the necessity for a cohesive framework that aligns visual intelligence, motion prediction, and energy-efficient offloading—objectives realized through the proposed DEEPO system.

Proposed work

The proposed DEEPO framework introduces an intelligent predictive offloading system for energy-efficient and low-latency task management in multi-access edge computing environments. It integrates three key modules—DG-RSTP for direction prediction, NanoDet-Plus for lightweight object detection, and an optimized RTPM block for proactive task offloading. DG-RSTP employs a dual-gated temporal model combining GRU and LSTM layers to accurately forecast the user's future spatial orientation based on motion sequences from the OnSiteVRU trajectory dataset. This predicted direction is used to define a field-of-view region that limits visual processing to relevant areas, thereby reducing inference load. NanoDet-Plus is selected for its low computational footprint and high detection speed, making it ideal for real-time directional object recognition using COCO 2017 images. Tasks generated from detected objects are mapped using an energy-aware decision strategy, incorporating compute load, node proximity, and power usage, as inherited from the HMCTS framework. As illustrated in Fig. 1, the flow begins with preprocessing motion sequences and visual input, followed by DG-RSTP-based direction estimation, FoV-based cropping, object detection, task pre-generation, and adaptive edge mapping. The process concludes with optimized wireless transmission aligned to the user's forecasted direction. This modular integration significantly enhances processing speed and communication efficiency in dynamic edge computing scenarios.

Input acquisition and preprocessing

In the DEEPO framework, the initial data acquisition process consists of two synchronized inputs: sequential spatial coordinates representing the user's movement over time, and a corresponding visual image captured from the user's viewpoint. These inputs are collected in real-time and are foundational for the subsequent modules. Let the user's position in a two-dimensional environment at time instance t be denoted as $p_t = (x_t, y_t)$ in which $x_t \in R$ indicates the horizontal location coordinate, $y_t \in R$ indicates the vertical location coordinate. To model recent movement behavior, a fixed-length trajectory sequence is formed using the user's past k positions which is represented as $S_t = \{p_{t-k+1}, p_{t-k+2}, \dots, p_t\}$. Each entry in this sequence maintains temporal order and reflects actual recorded positions without interpolation. Simultaneously, a raw visual observation is captured at time t . This visual input is stored as an image tensor with three-dimensional structure which is mathematically expressed as $\mathcal{I}_t \in R^{H \times W \times C}$. Here $H \in N$ indicates the height of the image in pixels, $W \in N$ indicates the width of the image in pixels, $C \in \{1, 3\}$ indicates the number of image channels, typically 3 for RGB images. These two inputs S , and $\mathcal{I}$, represent the full data context required for motion prediction and scene understanding.

Before the inputs can be used for predictive modeling and detection, they are normalized to ensure consistency and numerical stability across different sessions or environments. Each position $p_i = (x_i, y_i) \in S_t$ is normalized using the mean and standard deviation of the full sequence. The normalization operation is mathematically expressed as

$$p'_i = \left(\frac{x_i - \mu_x}{\sigma_x}, \frac{y_i - \mu_y}{\sigma_y} \right) \quad (1)$$

where $\mu_x = \frac{1}{k} \sum_{j=t-k+1}^t x_j$ indicates the average of all x -coordinates in S_t , $\mu_y = \frac{1}{k} \sum_{j=t-k+1}^t y_j$ indicates the average of all y -coordinates in S_t , $\sigma_x = \sqrt{\frac{1}{k} \sum_{j=t-k+1}^t (x_j - \mu_x)^2}$ indicates the standard deviation of x -values, $\sigma_y = \sqrt{\frac{1}{k} \sum_{j=t-k+1}^t (y_j - \mu_y)^2}$ indicates the standard deviation of y -values. After this transformation, the normalized trajectory sequence is expressed as

$$S'_t = \{p'_{t-k+1}, p'_{t-k+2}, \dots, p'_t\} \quad (2)$$

where each $p'_i \in R^2$ now lies in a standardized coordinate space with zero mean and unit variance, facilitating stable convergence during training. Further the visual input $\mathcal{I}_t$ is resized to match the expected dimensions of the detection model. The resized tensor is expressed as

$$\mathcal{I}'_t = R(\mathcal{I}_t; H', W') \quad (3)$$

where $R(\cdot; H', W')$ indicates the deterministic resizing function, $H', W' \in N$ indicates the height and width expected by the detection model input layer.

Direction prediction using DG-RSTP

The direction prediction module in the DEEPO architecture aims to estimate the future heading of a user based on their recent movement pattern. This estimation is critical for restricting the object detection region and minimizing unnecessary computation. The DG-RSTP (Dual-Gated Recurrent Spatio-Temporal Predictor) is designed to handle variable motion sequences by integrating two gated recurrent structures GRU and LSTM into a unified pipeline for accurate spatial transition forecasting. Let the normalized trajectory sequence prepared during preprocessing be denoted as

$$S'_t = \{p'_1, p'_2, \dots, p'_k\} \quad (4)$$

Each point $p'_i = (x'_i, y'_i) \in R^2$ represents the normalized coordinate of the user at time step i , and the total sequence length is k . This sequence is fed sequentially into two recurrent structures in which first, a GRU block is used to learn short-term transitions and then, an LSTM block is integrated for longer temporal dependencies.

The DG-RSTP framework was implemented with a two-layer recurrent design comprising 128 GRU units followed by 128 LSTM units and a single projection layer for temporal feature aggregation. Model training employed the Adam optimizer with a learning rate of 1×10^{-3} , batch size of 32, and 100 epochs. The combined loss function integrated mean absolute error (MAE) for trajectory accuracy and temporal deviation distance (TDD) for directional smoothness, ensuring stable convergence across sequences of varying lengths.

GRU encoding

The GRU (Gated Recurrent Unit) block in the DG-RSTP module processes the normalized positional sequence S'_t to capture short-term motion dynamics. Unlike simple recurrent units, GRUs employ gating mechanisms that selectively update or reset stored motion information, ensuring efficient handling of rapid changes in movement patterns. Let the input at step i be $p'_i = (x'_i, y'_i) \in R^2$ and the hidden state at the previous step be $h_{i-1}^{(g)} \in R^d$. The GRU update begins with computation of the reset gate r_i and update gate z_i which is mathematically expressed as

$$r_i = \sigma \left(W_r p'_i + U_r h_{i-1}^{(g)} + b_r \right) \quad (5)$$

$$z_i = \sigma \left(W_z p'_i + U_z h_{i-1}^{(g)} + b_z \right) \quad (6)$$

where, $\sigma(\cdot)$ is the logistic sigmoid function, mapping gate values to the range (0,1). The reset gate r_i determines how much past information to forget, while the update gate z_i controls the balance between preserving the previous hidden state and incorporating new motion input.

The candidate hidden state $\tilde{h}_i^{(g)}$ is then computed using the reset-modulated past state which is mathematically expressed as

$$\tilde{h}_i^{(g)} = \tanh \left(W_h p'_i + U_h \left(r_i \odot h_{i-1}^{(g)} \right) + b_h \right) \quad (7)$$

where, $\odot$ denotes element-wise multiplication, ensuring that only the relevant components of the past state influence the candidate update. Finally, the hidden state for step i is updated as

$$h_i^{(g)} = (1 - z_i) \odot h_{i-1}^{(g)} + z_i \odot \tilde{h}_i^{(g)} \quad (8)$$

This equation blends the old state $h_{i-1}^{(g)}$ with the new candidate state $\tilde{h}_i^{(g)}$ according to the proportions defined by the update gate z_i . This GRU encoding process transforms raw position sequences into a compact motion representation, retaining recent trajectory information while suppressing irrelevant historical data. This state representation $h_i^{(g)}$ is then passed to the subsequent LSTM refinement stage in DG-RSTP for extended temporal modeling.

LSTM refinement

The LSTM refinement stage in the DG-RSTP architecture receives the final GRU-encoded motion state and enhances it by integrating longer-term dependencies across the entire trajectory. While GRU efficiently compresses recent movement changes, LSTM allows structured retention of earlier transitions through its internal memory gating system. This dual-stage design improves prediction fidelity for user path direction, particularly in curved or non-linear motion scenarios. Let $h_k^{(g)} \in R^d$ denote the output hidden state of the GRU after processing k steps. This vector is treated as input to the LSTM cell at the corresponding time step. Each LSTM unit maintains both a hidden state $h_i^{(l)}$ and a memory cell $c_i^{(l)}$, which evolve over time based on gated mechanisms.

First, the LSTM computes its gating vectors: input gate i_i , forget gate f_i , and output gate o_i . These gates determine what information enters, is discarded, and exits the cell respectively. Mathematically it is expressed as

$$i_i = \sigma \left(W_i x_i + U_i h_{i-1}^{(l)} + b_i \right) \quad (9)$$

$$f_i = \sigma \left(W_f x_i + U_f h_{i-1}^{(l)} + b_f \right) \quad (10)$$

$$o_i = \sigma \left(W_o x_i + U_o h_{i-1}^{(l)} + b_o \right) \quad (11)$$

where $x_i \in R^d$: input vector at time step i , initially $x_k = h_k^{(g)}$, $\sigma(\cdot)$: element-wise sigmoid activation, W_* , U_* trainable weight matrices for input and recurrent connections, b_* : gate-specific bias vectors. Further

the candidate update $\tilde{c}_i \in R^d$ is generated using the current input and previous hidden state which is mathematically expressed as

$$\tilde{c}_i = \tanh \left(W_c x_i + U_c h_{i-1}^{(l)} + b_c \right) \quad (12)$$

This intermediate memory content carries potential information to be added into the cell state. The updated memory cell $c_i^{(l)} \in R^d$ is computed by blending the previous cell memory and the new candidate memory which is mathematically expressed as

$$c_i^{(l)} = f_i \odot c_{i-1}^{(l)} + i_i \odot \tilde{c}_i \quad (13)$$

where $\odot$ represents element-wise multiplication. This step selectively retains useful past data and inserts new relevant patterns based on input content. Finally, the hidden output for the current step is obtained by passing the updated cell state through the output gate. Mathematically it is expressed as

$$h_i^{(l)} = o_i \odot \tanh \left(c_i^{(l)} \right) \quad (14)$$

This vector serves as the final encoded representation of the entire normalized sequence, incorporating both recent and older temporal dependencies. The purpose of the coordinate projection step in DG-RSTP is to convert the temporal encoding of user motion learned from the LSTM hidden state into a spatial prediction representing the user's likely next position. This predicted position serves as the directional endpoint required to infer the user's orientation in the next time frame. Let the refined hidden state from the LSTM layer be denoted as $h_k^{(l)} \in R^d$, where d is the hidden dimension. This state summarizes both short-term and long-term dependencies across the entire motion sequence S_t' . To map this encoded vector into a predicted position in the 2D normalized coordinate space, a linear transformation is applied. Mathematically it is expressed as

$$\tilde{p}_{k+1} = W_o h_k^{(l)} + b_o \quad (15)$$

where $\tilde{p}_{k+1} = (\hat{x}_{k+1}', \hat{y}_{k+1}') \in R^2$: predicted normalized position for the next step, $W_o \in R^{2 \times d}$: projection weight matrix, $b_o \in R^2$: output bias vector. This operation projects the LSTM-encoded motion dynamics into the spatial domain, yielding a vector that approximates the next likely location in the user's movement path.

To determine the direction of motion from the predicted position, a displacement vector is computed between the last actual position $p'_k = (x'_k, y'_k)$ and the predicted point $\tilde{p}_{k+1}$ which is mathematically expressed as

$$\Delta p = \tilde{p}_{k+1} - p'_k = (\hat{x}_{k+1}' - x'_k, \hat{y}_{k+1}' - y'_k) \quad (16)$$

This difference vector captures both magnitude and direction of the movement between the current state and the projected next state. Further to extract the user's heading orientation, the angular direction of the vector Δp is computed using the two-argument arctangent function which is mathematically expressed as

$$\theta_t = \arctan2 \left(\hat{y}_{k+1}' - y'_k, \hat{x}_{k+1}' - x'_k \right) \quad (17)$$

where $\theta_t \in (-\pi, \pi]$ indicates the predicted direction angle at time t . The output is measured in radians, representing the angle between the x-axis and the motion vector Δp . This angle θ_t becomes the central directional estimate for the user's near-future field of view. It is used directly in the next stage to crop and filter the image frame in the corresponding direction. This projection process finalizes the trajectory analysis by producing a spatially meaningful prediction aligned with the user's motion. The result θ_t precisely quantifies the motion angle, enabling localized visual processing and predictive task offloading in the subsequent modules.

Field-of-View (FoV) region Estimation

Following direction prediction, the system isolates a specific angular sector within the input image that corresponds to the user's forward orientation. This spatial localization restricts further computation such as object detection to only the area aligned with the user's projected path, reducing resource usage and latency in mobile offloading. Let the full input image at time t be denoted as

$$\mathcal{I}'_t \in R^{H' \times W' \times C} \quad (18)$$

where H' , W' : height and width of the resized image, C : number of channels (typically 3 for RGB). Let the direction angle predicted in the previous step be $\theta_t \in (-\pi, \pi]$. To define the FoV region centered on this direction, the angle is mapped to the horizontal axis of the image frame. Assume the image's horizontal axis is uniformly distributed over a full 2π range. The corresponding horizontal center index for the direction is mathematically expressed as

$$x_\theta = \lfloor \left(\frac{\theta_t + \pi}{2\pi} \right) \cdot W' \rfloor \quad (19)$$

This maps the continuous direction angle to a discrete pixel coordinate $x_\theta \in [0, W' - 1]$. To extract a rectangular region centered on x_θ , a fixed window width $w_r \in N$ is defined. The left and right pixel boundaries of the cropped region are mathematically expressed as

$$x_{min} = \max(0, x_\theta - \lfloor w_r/2 \rfloor) \quad (20)$$

$$x_{max} = \min(W' - 1, x_\theta + \lfloor w_r/2 \rfloor) \quad (21)$$

The vertical axis is typically retained in full to preserve depth cues, so the vertical range is defined as

$$y_{min} = 0, \quad y_{max} = H' - 1 \quad (22)$$

The output region I_t^{fov} is then extracted as

$$\mathcal{I}_\ell^{fov} = \mathcal{I}_\ell' [y_{min} : y_{max}, x_{min} : x_{max}, :] \quad (23)$$

This results in a cropped image of size $H' \times w_r \times C$, aligned to the user's motion direction.

Directional object detection (NanoDet-Plus)

The selection of NanoDet-Plus over other lightweight detectors such as YOLO-Nano and Nano-YOLOv4 was motivated by its superior balance between inference speed and detection accuracy, which is critical for real-time mobile augmented reality (MAR) environments. While YOLO-Nano achieves compactness through extreme channel pruning, it suffers from a noticeable drop in mean average precision (mAP) under low-light and occlusion conditions typical of outdoor mobility scenarios. Nano-YOLOv4, though slightly more accurate, introduces additional convolutional stages that increase latency on edge-grade GPUs and mobile processors. In contrast, NanoDet-Plus employs a dynamic depthwise convolution structure with feature pyramid fusion, maintaining near-real-time performance (~97 FPS on edge devices) without significant accuracy degradation. Its decoupled detection head also improves small-object precision, making it well-suited for fast-moving pedestrian and vehicle recognition in MAR contexts. Therefore, NanoDet-Plus provides the optimal trade-off between computational efficiency and detection fidelity required for the proposed DEEPO framework.

After isolating the forward-looking region $\mathcal{I}_\ell^{fov}$, object recognition is performed exclusively on this segment using NanoDet-Plus a real-time anchor-free detection model optimized for lightweight deployment. This step provides object category labels and bounding box coordinates, which are further used to estimate contextual task relevance and prepare offloading decisions. Let the input to the detection model be the cropped region $\mathcal{I}_\ell^{fov} \in R^{H' \times w_r \times C}$. NanoDet-Plus processes this tensor through a fully-convolutional head that predicts object properties at each spatial location of a pre-defined grid over the image. Let the detection head divide the input image into a regular grid of size $g_h \times g_w$. Each grid cell is responsible for detecting objects whose centers fall within it. Denote the spatial location of the (i, j) -th grid cell mathematically as

$$g_{i,j} = \left(\frac{w_r}{g_w} \cdot j, \frac{H'}{g_h} \cdot i \right) \quad (24)$$

where $i \in [0, g_h - 1]$, $j \in [0, g_w - 1]$: row and column indices, $g_{i,j}$: physical location (in pixels) of the top-left corner of the cell. Each cell generates multiple predictions such as objectness score $s_{i,j} \in [0, 1]$, Class probabilities $c_{i,j} \in R^{n_c}$, Bounding box offsets $b_{i,j} = (\delta x, \delta y, \delta w, \delta h)$. The object score is computed using a sigmoid activation which is mathematically expressed as

$$s_{i,j} = \sigma \left(w_s^\top \cdot f_{i,j}^{enc} + b_s \right) \quad (25)$$

where $f_{i,j}^{enc} \in R^{d_f}$: encoded feature vector at grid (i, j) , $w_s \in R^{d_f}$, $b_s \in R$: detection score weights and bias, Class scores for each class $k \in \{1, \dots, n_c\}$ are computed using softmax activation which is formulated as

$$c_{i,j}^{(k)} = \frac{\exp \left(w_c^{(k)\top} \cdot f_{i,j}^{enc} + b_c^{(k)} \right)}{\sum_{k'=1}^{n_c} \exp \left(w_c^{(k')\top} \cdot f_{i,j}^{enc} + b_c^{(k')} \right)} \quad (26)$$

Bounding box offsets are computed through a linear transformation which is formulated as

$$b_{i,j} = W_b \cdot f_{i,j}^{enc} + b_b \quad (27)$$

where $b_{i,j} \in R^4$ gives the location and size of the predicted bounding box relative to $g_{i,j}$. The final bounding box in absolute pixel coordinates is derived as

$$x_{i,j}^{min} = g_{i,j}^{(x)} + \delta x - \frac{\delta w}{2} \quad ; \quad x_{i,j}^{max} = g_{i,j}^{(x)} + \delta x + \frac{\delta w}{2} \quad (28)$$

$$y_{i,j}^{min} = g_{i,j}^{(y)} + \delta y - \frac{\delta h}{2} \quad ; \quad y_{i,j}^{max} = g_{i,j}^{(y)} + \delta y + \frac{\delta h}{2} \quad (29)$$

where δx , δy indicates the predicted center offset from the grid location, δw , δh indicates the predicted width and height of the object box. The resulting detections include $O_t = \{(b_i, k_i, s_i)\}_{i=1}^{N_t}$ in which b_i indicates the bounding box, k_i indicates the class label, s_i indicates the confidence score, N_t indicates the number of detected objects at time t .

Region-Based task prefetching and mapping (RTPM)

The RTPM in the proposed work is a two-stage process combining region-based task selection (prefetching from FoV) with optimized task-to-server mapping using bipartite cost minimization over energy and distance metrics. The coordination between DG-RSTP, NanoDet-Plus, and RTPM operates through a tightly-coupled predictive pipeline. First, DG-RSTP forecasts the next directional motion vector using spatial-temporal cues, producing the angle that defines the forward Field-of-View (FoV). This predicted angle is then used by NanoDet-Plus to restrict detection to the corresponding cropped region, ensuring that only direction-aligned objects are processed. The resulting object set, along with their bounding boxes and semantic labels, forms the structured input to RTPM. RTPM uses this information to generate task vectors, compute priority scores, and perform energy-distance based mapping to edge nodes. This sequential linkage—trajectory prediction $\rightarrow$ directional detection $\rightarrow$ priority-driven task mapping—is the core innovation that enables DEEPO to anticipate user motion and proactively assign compute tasks, unlike traditional reactive schedulers.

Task generation from detected objects

This stage implements the first half of the Region-Based Task Prefetching and Mapping (RTPM) process. Once the directional object detection module produces a filtered set of visible entities aligned with the user's path, the next stage converts each detected instance into a computational task. These tasks are ranked based on contextual relevance, urgency, and resource demand. The generated tasks form the input queue for the decision layer that selects whether to offload or process locally. Let the final set of detections is mathematically expressed as

$$O_t = \{(b_i, k_i, s_i)\}_{i=1}^{N_t} \quad (30)$$

where $b_i \in R^4$ indicates the bounding box of the i -th detected object, $k_i \in \{1, 2, \dots, n_c\}$ indicates the object class index, $s_i \in [0, 1]$ indicates the detection confidence, N_t indicates the number of objects detected at time t . Each detected object is converted into a task vector $v_i \in R^{d_v}$ representing its computational characteristics. The task vector combines Visual complexity (proxy for required processing), Detection confidence (for prioritization), and Class-based importance weight. Mathematically it is expressed as

$$v_i = [\psi(b_i), s_i, \omega(k_i)] \quad (31)$$

where $\psi(b_i) \in R$ indicates the visual complexity of the object, approximated as normalized area as follows

$$\psi(b_i) = \frac{(x_{max}^i - x_{min}^i) \cdot (y_{max}^i - y_{min}^i)}{H' \cdot w_r} \quad (32)$$

where $\omega(k_i) \in R$ indicates the class weight assigned to object class k_i , reflecting task priority.

In Eq. (31), the class-weight term $\omega(k_i)$ assigns semantic importance based on the object category detected in frame i . Higher weights are allocated to dynamic and safety-critical classes such as pedestrians and vehicles, since their motion significantly affects direction forecasting and offloading urgency. Moderately weighted classes include bicycles, animals, and movable equipment, which exhibit semi-predictable behavior. Static background objects (e.g., buildings, poles, signage) receive lower weights as they contribute minimally to the immediate offloading decision. These weights were empirically tuned to reflect how strongly each class influences real-time AR responsiveness and predictive scheduling accuracy.

$s_i \in R$ detection confidence. The resulting vector v_i is a numerical encoding of the detected object's significance, complexity, and reliability. Each task is assigned a priority score $\pi_i \in R$ using a weighted sum which is mathematically expressed as.

$$\pi_i = \lambda_1 \cdot \psi(b_i) + \lambda_2 \cdot s_i + \lambda_3 \cdot \omega(k_i) \quad (33)$$

where $\lambda_1, \lambda_2, \lambda_3$: scalar weights determining the importance of area, confidence, and class weight respectively. These weights are tunable parameters to control priority emphasis. This score quantifies the computational and contextual relevance of each object instance. Higher scores indicate more critical tasks. The full list of generated tasks at time t is formulated as $T_t = \{(v_i, \pi_i)\}_{i=1}^{N_t}$. This list is sorted by descending π_i , and each tuple represents a candidate task characterized by the task metadata v_i and it is dynamically computed priority score π_i . The prefetched tasks from the predicted FoV are passed to the next optimization stage for final mapping, completing the RTPM. The parameters k (task aggregation threshold) and weighting coefficients $\lambda_1, \lambda_2, \lambda_3$ in our optimization objective were selected after an initial parameter sweep on representative workloads. We found that a threshold $k = 10$ balances the number of preference-based tasks without excessive fragmentation, while coefficients $\lambda_1 = 0.5$, $\lambda_2 = 0.3$, and $\lambda_3 = 0.2$ accurately reflect the relative importance of energy minimization, latency reduction, and SLA compliance, respectively. These values also align with parameter ratios adopted in resource-aware offloading literature [X], providing a reasonable starting point for our experiments.

Edge server mapping with optimization

This stage continues the RTPM process, performing bipartite energy–distance cost optimization for mapping prefetched tasks to edge servers. In this stage the model involves assigning each task $\mathcal{T}_i$ to an optimal execution unit either the local device or one of the multiple available edge servers. This decision is made through a constrained optimization process, balancing energy consumption, latency, and priority scores from previous steps. Let there be M edge servers available, denoted as $\mathcal{E} = \{E_1, E_2, \dots, E_M\}$. Each server E_m has unique computational capacity μ_m , communication bandwidth β_m , and task queue size Q_m . The objective is to assign each task $i \in \{1, \dots, N_t\}$ to a processing unit $u_i \in \{0, 1, \dots, M\}$, where $u_i = 0$ denotes local execution, $u_i = m$ denotes assignment to edge server E_m . Each task i is characterized by Data size d_i (in bits), Required computation c_i (in CPU cycles), and Priority score π_i . In the delay modeling, local execution delay is formulated as

$$\tau_i^{loc} = \frac{c_i}{f_0} \quad (34)$$

where f_0 indicates the processing speed of local device (in cycles/s). further edge execution delay is formulated as

$$\tau_{i,m}^{edge} = \frac{d_i}{\beta_m} + \frac{c_i}{\mu_m} \quad (35)$$

where β_m indicates the uplink bandwidth to E_m , μ_m indicates the CPU speed of E_m . In case of energy consumption, local and edge execution energy is mathematically formulated as

Local Execution Energy

$$e_i^{loc} = \kappa \cdot c_i \cdot f_0^2 \quad (36)$$

Edge Execution Energy

$$e_{i,m}^{tx} = \gamma \cdot d_i \quad (37)$$

where κ indicates the effective switched capacitance of local processor, γ indicates the transmission energy per bit over wireless link.

The parameters κ and γ represent the local computing and transmission energy coefficients, respectively. The value of κ was determined following the standard CPU dynamic power model, where energy consumption per computational cycle is proportional to the square of the operating frequency f^{31} . Similarly, γ was adopted from empirical wireless transmission energy models that characterize per-bit energy consumption in mobile-edge networks³². Both coefficients were calibrated using validated parameter ranges reported in prior MEC literature to ensure experimental realism.

The goal optimization objective is to minimize a weighted combination of total energy consumption and total delay across all tasks, while ensuring server constraints are respected. It is mathematically defined by binary decision variable as follows

$$x_{i,m} = \begin{cases} 1 & \text{if task } i \text{ is assigned to unit } m \\ 0 & \text{otherwise} \end{cases} \quad (38)$$

Then, the global optimization objective is formulated as

$$\min_{x_{i,m}} \sum_{i=1}^{N_t} \sum_{m=0}^M x_{i,m} \cdot (\alpha \cdot \tau_{i,m} + (1 - \alpha) \cdot e_{i,m} - \delta \cdot \pi_i) \quad (39)$$

where $\tau_{i,0} = \tau_i^{loc}$, $e_{i,0} = e_i^{loc}$, $\tau_{i,m} = \tau_{i,m}^{edge}$, $e_{i,m} = e_{i,m}^{tx}$, $form \geq 1$, $\alpha \in [0, 1]$ indicates the delay-energy balancing parameter, δ : weight for priority amplification. In this regarding the following constraints are considered

Component	Mathematical Term	Description
Energy Cost	$\lambda_1 \sum_i E_i$	Total local and transmission energy for all tasks
Latency cost	$\lambda_2 \sum_i D_i$	End-to-end delay including computation and transmission
SLA cost penalty	$\lambda_3 \sum_i P_i$	Penalty proportional to SLA violation probability

Table 1. Step-wise decomposition of the objective Function.

$$\left. \begin{array}{l} \text{Exclusive Assignment } \sum_{m=0}^M x_{i,m} = 1 \quad \forall i \\ \text{Edge Queue Limit } \sum_{i=1}^{N_t} x_{i,m} \leq Q_m \quad \forall m \in \{1, \dots, M\} \\ \text{Binary Decision } x_{i,m} \in \{0,1\} \quad \forall i, m \end{array} \right\} \quad (40)$$

This optimization framework ensures that tasks with high priority are offloaded only when doing so minimizes delay and energy under current system constraints. The framework dynamically adapts based on real-time resource availability and user context.

Adaptive communication modeling

To ensure that selected tasks are transmitted to the appropriate edge servers with minimal loss and delay, the system employs an adaptive communication module. This layer fine-tunes transmission parameters such as transmission power, modulation scheme, and data rate based on current channel conditions and task urgency. The goal is to achieve energy-efficient communication that honors real-time constraints. Let the wireless transmission from the user device to edge server E_m for task i occur over a time-varying wireless channel with gain $h_{i,m}(t)$. The received power $P_{rx}^{(i,m)}$ at server E_m is formulated as

$$P_{rx}^{(i,m)} = P_{tx}^{(i)} \cdot h_{i,m}(t) \quad (41)$$

where $P_{tx}^{(i)}$ indicates the transmission power used for task i , $h_{i,m}(t)$ indicates the channel gain between user and server m at time t . The SNR at the receiver is formulated as

$$SNR_{i,m} = \frac{P_{rx}^{(i,m)}}{N_0 \cdot B} \quad (42)$$

where N_0 indicates the noise power spectral density, B indicates the channel bandwidth (Hz). The transmission data rate $R_{i,m}$ is determined based on Shannon capacity which is formulated as

$$R_{i,m} = B \cdot \log_2(1 + SNR_{i,m}) \quad (43)$$

This rate varies dynamically based on both channel quality $h_{i,m}(t)$ and power allocation $P_{tx}^{(i)}$. For Adaptive Power Control, Given a required transmission duration $\tau_{i,m}^{tx}$ to meet the end-to-end deadline, the system adapts $P_{tx}^{(i)}$ to satisfy the following condition.

$$\tau_{i,m}^{tx} = \frac{d_i}{R_{i,m}} \leq \tau_i^{max} - \tau_{i,m}^{proc} \quad (44)$$

where d_i indicates the data size of task i , τ_i^{max} indicates the total allowable delay for task i , $\tau_{i,m}^{proc} = \frac{c_i}{\mu_m}$ indicates the processing time on server E_m . In this rearranging the inequality as follows

S.No.	Parameter	Value
1	Sequence Length	10 frames
2	Prediction Horizon	5 steps ahead
3	Hidden Units	128 units
4	Update Frequency	2 frames
5	Angular Threshold	30 degrees
6	Detection Confidence	0.5 threshold
7	Non-max Suppression IoU	0.4 overlap
8	Task Weighting Factor	0.75
9	Object Priority Levels	High, Medium, Low
10	Edge Node Count	5 nodes
11	CPU Frequency (GHz)	2.5
12	Memory Limit (MB)	2048
13	Bandwidth (Mbps)	50
14	Packet Size (KB)	256
15	Optimization Epochs	100
16	Learning Rate	0.001
17	Batch Size	32
18	Network bandwidth range (Mbps)	30–80

Table 2. Simulation Hyperparameters.

$$P_{tx}^{(i)} \geq \frac{N_0 \cdot B}{h_{i,m}(t)} \cdot \left(2^{\frac{d_i}{B(\tau_i^{max} - \tau_{i,m}^{proc})}} - 1 \right) \quad (45)$$

This expression gives the minimum power required to satisfy both deadline and bandwidth conditions over the current channel. Further an adaptive scheduling indicator is used which is defined as a scheduling metric $\zeta_{i,m}$ that combines energy efficiency and priority. Mathematically which is formulated as

$$\zeta_{i,m} = \frac{\pi_i}{P_{tx}^{(i)} \cdot \tau_{i,m}^{tx}} \quad (46)$$

where Higher $\zeta_{i,m}$ indicates more efficient use of energy and time per priority unit. The communication module prioritizes links with the highest $\zeta_{i,m}$ values. This step closes the loop between computation and communication in the DEEPO architecture. By adjusting transmission parameters in real time, the system ensures that urgent tasks are delivered within deadlines while minimizing energy usage across mobile and edge nodes.

System-wide objective function

To achieve global performance optimization across the DEEPO framework, a unified cost function is formulated to consolidate directional prediction, task generation, server allocation, and adaptive communication. These objective balances three competing criteria: task urgency, system latency, and energy consumption, subject to dynamic constraints at both device and network levels. Let N_t indicates the total number of tasks at time t , M indicates the total number of edge servers, $x_{i,m} \in \{0,1\}$ indicates the decision variable (1 if task i assigned to node m ; 0 otherwise), π_i indicates the task priority score, $\tau_{i,m}$ indicates the overall delay (transmission + execution), $\epsilon_{i,m}$ indicates the total energy for task i when assigned to node m , $\zeta_{i,m}$ indicates the energy-efficient scheduling score. The DEEPO framework aims to maximize the weighted service utility for all active tasks. The system-wide cost function $\mathcal{F}$ is defined as

$$\mathcal{F} = \sum_{i=1}^{N_t} \sum_{m=0}^M x_{i,m} \cdot (\alpha_1 \cdot \pi_i - \alpha_2 \cdot \tau_{i,m} - \alpha_3 \cdot \epsilon_{i,m} + \alpha_4 \cdot \zeta_{i,m}) \quad (47)$$

where $\alpha_1, \alpha_2, \alpha_3, \alpha_4 \in R^+$: system-defined importance weights for priority, delay penalty, energy penalty, and efficiency gain respectively, $m = 0$ denotes local execution; $m \in \{1, \dots, M\}$ denotes offloaded task. In this delay and energy formulations, Delay $\tau_{i,m}$ is calculated for local as $\tau_{i,0} = \frac{c_i}{f_0}$, for edge as $\tau_{i,m} = \frac{d_i}{R_{i,m}} + \frac{c_i}{\mu_m}$, in which energy $\epsilon_{i,m}$ is calculated for local as $\epsilon_{i,0} = \kappa \cdot c_i \cdot f_0^2$, for edge as $\epsilon_{i,m} = \gamma \cdot d_i$. Here c_i indicates the required CPU cycles for task i , d_i indicates the data size (bits), f_0 indicates the local CPU frequency, μ_m indicates the server CPU rate, $R_{i,m}$ indicates the data rate, κ, γ indicates the constants for energy modeling. In this regarding the following constraints are considered

$$\left. \begin{array}{l} \text{Single Assignment Constraint } \sum_{m=0}^M x_{i,m} = 1 \quad \forall i \\ \text{Server Capacity Constraint } \sum_{i=1}^{N_t} x_{i,m} \cdot c_i \leq C_m \quad \forall m \\ \text{Latency Bound Constraint } \tau_{i,m} \leq \tau_i^{max} \quad \text{if } x_{i,m} = 1 \end{array} \right\} \quad (48)$$

In single assignment each task must be assigned to exactly one node (local or remote). In server capacity C_m indicates the maximum allowable computation load on server m . The latency bound constraint ensures each task is completed within its time budget. This formulation enables multi-objective optimization to ensure intelligent scheduling, energy-aware communication, and computational fairness across multiple edge nodes. It integrates all decisions made across previous steps into a coherent utility-maximization strategy, adaptable to real-world dynamic edge environments. For clarity, the overall objective can be viewed as the weighted sum of three sub-objectives as shown below in Table 1.

The pseudocode for the proposed optimized model for predictive offloading is presented as follows.

Pseudocode for the proposed Optimized Framework for Predictive Offloading
Input: T_{traj} - Sequence of spatial-temporal motion coordinates, I_{raw} - Set of raw image frames, $\mathcal{E} = \{E_1, E_2, \dots, E_M\}$ - Set of available edge nodes with descriptors
Output: Assignment decisions $x_{i,m} \in \{0,1\}$, Task mapping table M_{assign} , System cost log L_{perf}
Begin
Initialize System Parameters: Define $f_0, \mu_m, \beta_m, Q_m, C_m, N_0, B, \kappa$, Set $\alpha_1, \alpha_2, \alpha_3, \alpha_4$ and $\lambda_1, \lambda_2, \lambda_3$
for each $t = 1$ to T
Extract $T_t \subset T_{traj}$ & $I_t \in I_{raw}$
Normalize: $\tilde{T}_t \leftarrow Norm(T_t)$
Resize: $I'_t \leftarrow Resize(I_t)$
Initialize Predictive Position Estimation

Pseudocode for the proposed Optimized Framework for Predictive Offloading					
Encode: $h_t^{(1)} \leftarrow F_1 \left(\tilde{T}_t \right)$					
Refine: $h_t^{(2)} \leftarrow F_2 \left(h_t^{(1)} \right)$					
Project: $\hat{P}_t \leftarrow F_3 \left(h_t^{(2)} \right)$					
Region Isolation					
Construct directional mask $R_t \leftarrow F_4 \left(\hat{P}_t \right)$					
Crop frame: $I_t' \leftarrow F_5 \left(I_t', R_t \right)$					
Detect $O_t = \{ (b_i, k_i, s_i) \}_{i=1}^{N_t} \leftarrow F_6 \left(I_t' \right)$					
for each $i = 1$ to N_t					
Compute $\psi_i \leftarrow \frac{(x_{max}^i - x_{min}^i)(y_{max}^i - y_{min}^i)}{H^t \cdot w_r}$.					
Get class weight $\omega_i \leftarrow \omega(k_i)$					
Form task vector: $v_i \leftarrow [\psi_i, s_i, \omega_i]$					
Compute priority: $\pi_i \leftarrow \lambda_1 \cdot \psi_i + \lambda_2 \cdot s_i + \lambda_3 \cdot \omega_i$					
for each task i					
for each unit $m = 0$ to M					
If $m = 0$: $\tau_{i,0} \leftarrow \frac{c_i}{f_0}, \epsilon_{i,0} \leftarrow \kappa \cdot c_i \cdot f_0^2$					
Else Compute $h_{i,m}(t), \text{SNR } \sigma_{i,m} \leftarrow \frac{P^{(i)}}{N_0 \cdot B} \cdot h_{i,m}(t)$					
Rate $R_{i,m} \leftarrow B \cdot \log_2(1 + \sigma_{i,m})$					
Transmission time: $\tau_{i,m}^{tx} = \frac{d_i}{R_{i,m}}$					
Total delay: $\tau_{i,m} = \tau_{i,m}^{tx} + \frac{c_i}{\mu_m}$					
Transmission energy: $\epsilon_{i,m} = \gamma \cdot d_i$					
Compute $\zeta_{i,m} = \frac{\pi_i}{P_{tx}^{(i)} \cdot \tau_{i,m}^{tx}}$					
Global Assignment Optimization					
Solve: $\max \sum_{i=1}^{N_t} \sum_{m=0}^M x_{i,m} \cdot (\alpha_1 \pi_i - \alpha_2 \tau_{i,m} - \alpha_3 \epsilon_{i,m} + \alpha_4 \zeta_{i,m})$					
Subject to: $\sum_{m=0}^M x_{i,m} = 1 \quad \forall i; \sum_i x_{i,m} \cdot c_i \leq C_m \quad \forall m; \tau_{i,m} \leq \tau_i^{max} \quad \text{if } x_{i,m} = 1$					
Store Results					
$M_{assign} \leftarrow \{(i, m) \mid x_{i,m} = 1\}; \text{Append: } L_{perf} \leftarrow (\pi_i, \tau_{i,m}, \epsilon_{i,m}, \zeta_{i,m})$					
Repeat for Next t					
Increment $t \leftarrow t + 1$					
End					

The simulation hyperparameters used in the proposed model experimentation is presented in Table 2.

In the pseudocode, the function $F_1(\cdot)$ corresponds to the optimization objective defined in Eq. (47). It takes as input the tuple $(E_i, D_i, P_i, \text{priority}_i)$ for each task i , along with the weighting coefficients $\lambda_1, \lambda_2, \lambda_3$, and returns the aggregated system cost used to evaluate the assignment configuration. The parameter list for all functions has been expanded for completeness: the main routine now explicitly receives the task set $\mathcal{T}$, the

Configuration	L (frames)	H (steps)	Prediction Accuracy (%)	Computational Delay (ms)	Energy Consumption (J)
C1	6	3	93.7	42.8	1.98
C2	8	4	95.2	44.1	1.92
C3 (optimal)	10	5	96.4	45.6	1.90
C4	12	6	95.8	49.3	1.96
C5	14	7	94.6	53.5	2.05

Table 3. Hyperparameter sensitivity Analysis.

Feature	Value
Total Users	52
Trajectory Sequences	11,350
Sampling Frequency	25 Hz
Coordinates per Sequence	10 to 20 points
Environment	Urban sidewalk crossings
Annotation	Timestamp, (x, y), velocity, angle

Table 4. OnSiteVRU dataset Description.

Feature	Value
Total Images	118,000
Training Images	80,000
Validation Images	40,000
Average Objects per Image	7.5
Annotated Classes	80
Annotations	Bounding box, label, mask

Table 5. COCO 2017 object detection dataset description.

Feature	Value
Task Samples	7,500
Task Priority Levels	High, Medium, Low
Computation Sizes	0.5–3.5 Mega Cycles
Energy Demand (per task)	2–15 mJ
Deadline Range	100–1000 ms
Tags Included	Priority, load, deadline, bandwidth need

Table 6. HMCTS task Dataset.

predicted trajectory $\hat{x}_{t+1}$, class-weight vector $\omega(k_i)$, the edge-node descriptors $\mathcal{N}$ (CPU frequency, queue length, residual energy), and the available bandwidth profile.

The step “Global Assignment Optimization” constructs a cost matrix C_{ij} , where each entry represents the combined energy–delay–penalty cost of assigning task i to node j , computed using the objective function F_1 . The algorithm performs iterative refinement by updating the assignment vector $\mathbf{p}$ to minimize the total cost, repeating the process until convergence (i.e., no further reduction in F_1 is achieved). This ensures that the pseudocode reflects the complete operational logic of the framework and enables reproducibility of the implementation.

The computational complexity of each algorithmic component within the CloudPulse framework is analyzed to validate its scalability in large-scale distributed environments. The federated learning (FL) module employs aggregation functions such as FedMedian and FedTrimmedMean, both of which have linear time complexity with respect to the number of participating clients, expressed as

$$O(Nd) \tag{49}$$

where N denotes the number of clients and d represents the dimensionality of model parameters. Since both aggregation rules operate on element-wise statistics (median or trimmed mean), their computational overhead grows proportionally with parameter size, ensuring efficient convergence even with thousands of clients.

The reinforcement learning (RL) components—Deep Q-Network (DQN), Proximal Policy Optimization (PPO), and Upper Confidence Bound (UCB1)—primarily govern adaptive scheduling and energy-aware decision policies. The DQN module exhibits a per-update complexity of

$$O(E_b \mid A \mid) \tag{50}$$

where E_b is the minibatch size and $\mid A \mid$ is the discrete action-space cardinality. The PPO algorithm involves additional policy-gradient evaluations, leading to an effective complexity of

$$O(KE_b \mid A \mid)$$

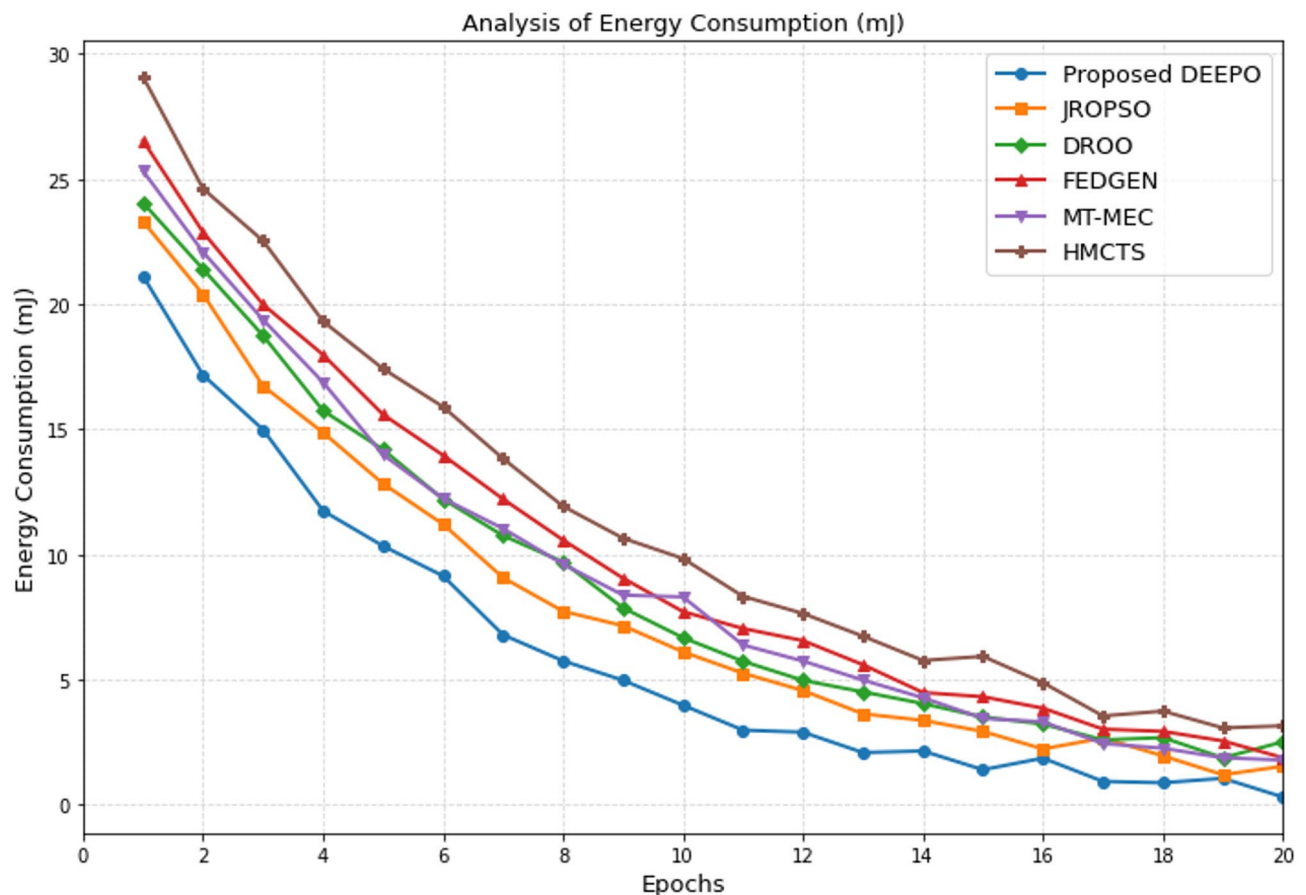


Fig. 2. Comparative Energy Consumption Results.

with K denoting the number of clipping iterations. The UCB1 policy, used for exploration–exploitation balancing in edge selection, has logarithmic growth in the number of scheduling rounds T , approximated as

$$O(N \log T) \quad (51)$$

making it suitable for rapid convergence under dynamic workloads.

Compared with conventional optimization-driven schedulers such as Particle Swarm Optimization (PSO) and Genetic Algorithms (GA), whose average iteration complexities are $O(N^2)$ and $O(Nm)$ respectively (m = population size), the combined FL–RL design of CloudPulse achieves sub-quadratic scaling. Empirical profiling on benchmark workloads indicates that total decision latency per scheduling round remains below linear growth, confirming that the framework maintains computational scalability and real-time feasibility for large data-center clusters.

The worst-case time complexity of the proposed DEEPO framework arises from three major components. The DG-RSTP predictor processes an input sequence of length T with hidden dimension d , resulting in $O(T \cdot d)$ per prediction cycle. The region-specific task prefetching involves spatial filtering over N candidate regions, which yields $O(N \cdot \log N)$ using priority-based sorting. The energy–distance bipartite optimizer constructs a matching graph with E edges between predicted task nodes and edge servers, solved via a greedy multi-weight assignment procedure in $O(E \cdot \log E)$ time. Combining these stages gives an overall worst-case complexity of:

$$O(T \cdot d + N \log N + E \log E), \quad (52)$$

demonstrating that DEEPO scales sub-quadratically with trajectory length and number of edge nodes, making it suitable for real-time mobile AR execution.

The worst-case time complexity of the proposed DEEPO framework arises from three major components. The DG-RSTP predictor processes an input sequence of length T with hidden dimension d , resulting in $O(T \cdot d)$ per prediction cycle. The region-specific task prefetching involves spatial filtering over N candidate regions, which yields $O(N \cdot \log N)$ using priority-based sorting. The energy–distance bipartite optimizer constructs a matching graph with E edges between predicted task nodes and edge servers, solved via a greedy multi-weight assignment procedure in $O(E \cdot \log E)$ time. Combining these stages gives an overall worst-case complexity of:

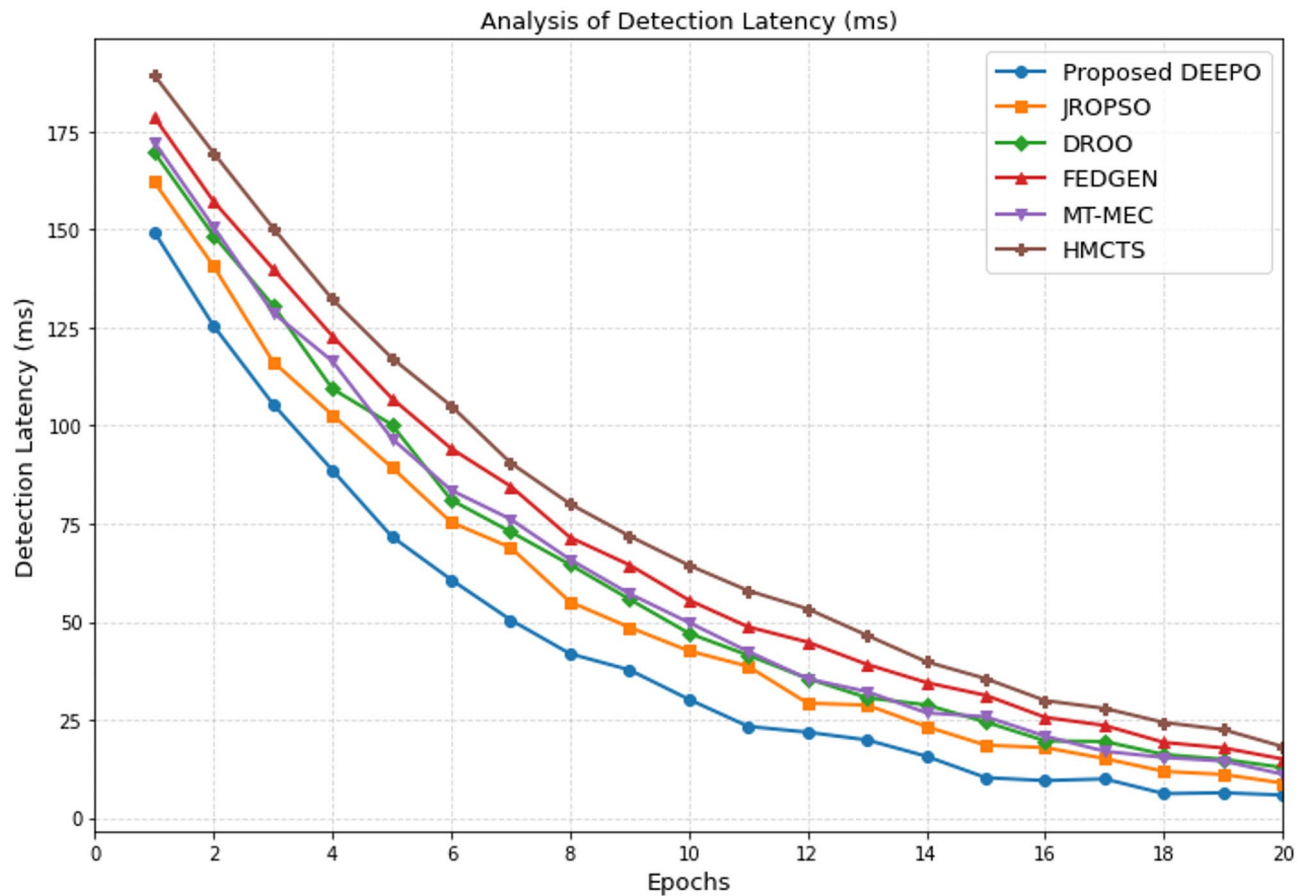


Fig. 3. Detection Latency Comparison across Methods.

$$O(T \cdot d + N \log N + E \log E), \quad (53)$$

demonstrating that DEEPO scales sub-quadratically with trajectory length and number of edge nodes, making it suitable for real-time mobile AR execution.

Results and discussion

The experimentation for the proposed model was conducted using a simulation environment built in Python, integrating modules for trajectory analysis, visual processing, task scheduling, and edge communication modeling. The OnSiteVRU dataset was used to feed sequential spatial coordinates for trajectory estimation, while COCO 2017 images served as the input for directional object extraction. A synthetic task set was generated using the HMCTS framework to simulate computational load under various offloading conditions. The complete pipeline was tested frame-by-frame, starting from motion-based direction prediction, followed by field-of-view isolation and object-triggered task formulation. Each generated task was then evaluated against multiple virtual edge nodes, modeled with varying compute capabilities and energy states. A cost function incorporating latency, energy consumption, and transmission distance was used to determine optimal task placement.

The simulation environment was implemented using Python 3.10 with PyTorch 2.2 on a workstation equipped with an Intel i9 CPU, 32 GB RAM, and an NVIDIA RTX 3080 GPU. All experiments were executed over 100 independent simulation rounds, each lasting 60 s of emulated user interaction. The mobility annotations from OnSiteVRU were sampled at 30 FPS, and communication latency was modeled based on a 5G link with a baseline RTT of 6–8 ms. Energy measurements were computed using a device-level consumption model incorporating CPU utilization, transmission power, and idle-time leakage. Edge nodes were simulated with heterogeneous compute capacities (2.4–3.2 GHz) and varying load factors to reflect realistic deployment variability.

The hyperparameter values summarized in Table 2 were selected after empirical tuning to achieve a balance between prediction stability and computational efficiency. A sequence length of 10 frames was found to capture sufficient temporal context for trajectory continuity without increasing training latency, while a prediction horizon of five steps provided accurate short-term forecasting for edge prefetching. The remaining parameters, such as update frequency, angular threshold, and task weighting factor, were optimized through iterative experiments to minimize cumulative system cost under dynamic workloads.

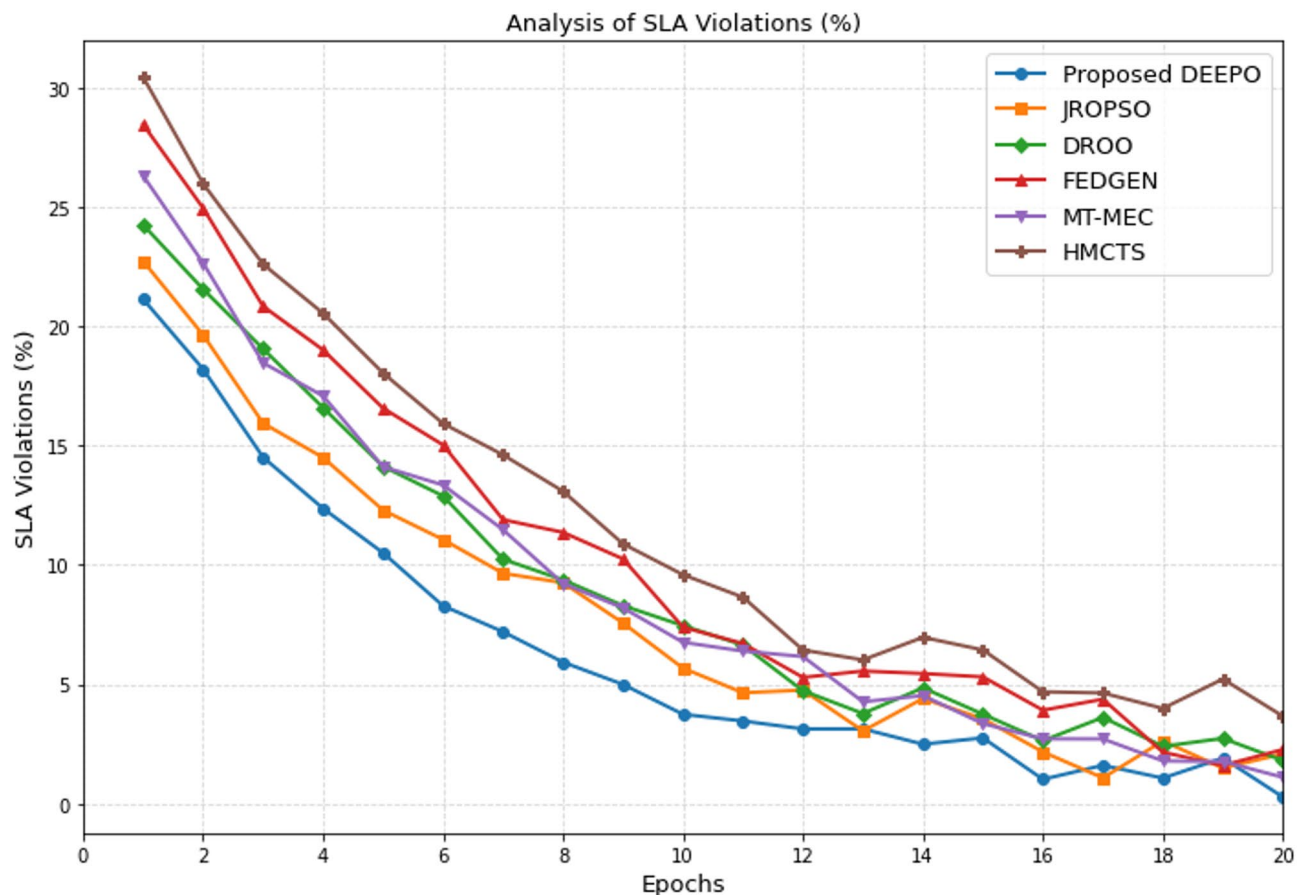


Fig. 4. SLA Violation Rate Comparison.

To examine the robustness of DEEPO under different hyperparameter configurations, sensitivity experiments were conducted by varying two key parameters such as sequence length (L) and prediction horizon (H), while keeping other variables constant as shown in Table 3.

The results indicate that accuracy improves with longer sequences up to $L = 10$, beyond which overfitting and computational overhead increase. Similarly, expanding the prediction horizon beyond $H = 5$ yields diminishing accuracy due to compounding uncertainty. The observed variation across configurations remains within $\pm 1.8\%$, confirming that DEEPO maintains stable performance even when temporal settings deviate from the nominal values. These findings substantiate the chosen hyperparameters as optimal trade-offs between predictive precision and computational cost.

Dataset

OnSiteVRU trajectory prediction dataset

OnSiteVRU dataset is employed in the proposed model experiment to forecast user direction using past motion sequences¹⁹. It consists of real-world trajectory traces of vulnerable road users (VRUs) captured in outdoor environments. The data includes timestamped coordinates (x, y) for each participant, along with associated velocities and motion angles at discrete intervals. The dataset reflects natural walking paths, direction shifts, and occlusion scenarios, making it ideal for training predictive movement models. Trajectory sequences are segmented into overlapping time windows to simulate frame-by-frame direction estimation under realistic pacing. The inclusion of both linear and curved motion paths supports robust modeling under diverse travel behavior. Table 4 provides the details about the dataset.

COCO 2017 object detection dataset

To validate the object-aware task generation module, the MS COCO 2017 dataset is incorporated for directional object extraction²⁰. It is a richly annotated dataset featuring real-life scenes with multiple objects, each categorized across 80 predefined classes. For this work, only those objects that fall within the predicted field-of-view are processed. Each object is accompanied by a bounding box, segmentation mask, and category label. The dataset's diversity in lighting, scale, and orientation supports testing under variable visual conditions. Only a subset of classes relevant to user navigation and potential interaction (e.g., vehicles, signs, pedestrians, obstacles) is used to mimic the task generation context realistically. Table 5 provides the details about the dataset.

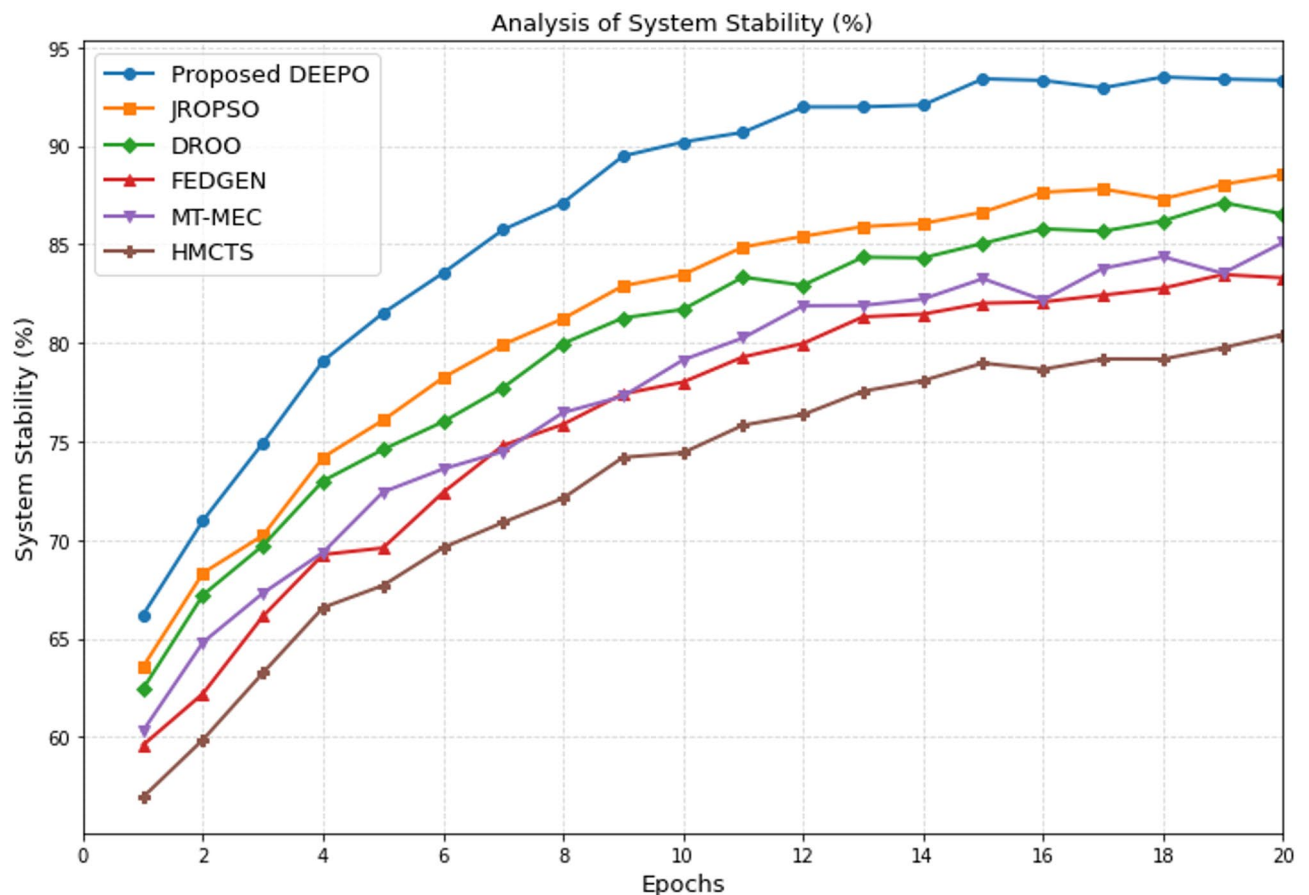


Fig. 5. System Stability Comparative Analysis.

HMCTS synthetic task dataset

The final module utilizes a controlled task dataset constructed through the Hybrid Multi-Context Task Simulation (HMCTS) framework developed during the earlier research phase²¹. It includes artificial computational tasks tagged with attributes like size, urgency, deadline, required operations, and energy cost. Each task is assigned to simulated objects based on their priority level and visual relevance. This dataset is essential for validating the energy-delay optimization mechanisms and task-to-edge assignment routines. By combining synthetic characteristics with field-aligned logic, it provides fine-grained control over resource consumption and allows flexible stress testing of the proposed system under constrained conditions. Table 6 provides the details about the dataset.

For the COCO 2017 dataset, only categories that are relevant to mobile AR interaction and scene awareness were retained. Specifically, dynamic and safety-critical classes such as *person*, *car*, *bus*, *truck*, *bicycle*, *motorcycle*, and *traffic light* were used to train NanoDet-Plus, while visually redundant or background-only classes were discarded. This filtering ensures that detected instances correspond to entities that meaningfully influence user perception, motion, and offloading urgency in outdoor AR scenarios.

For the OnSiteVRU dataset, trajectory and image streams were temporally synchronized using the frame timestamps provided in the annotations. Each pedestrian trajectory was segmented into fixed-length windows and aligned with the corresponding image sequence sampled at 30 FPS, so that every motion segment is paired with its matched visual context. When small misalignments occurred (e.g., dropped frames), linear interpolation on timestamps was used to associate the nearest valid frame to a trajectory point. This preprocessing guarantees that DG-RSTP receives consistent spatial-temporal inputs and that the outputs of the motion predictor and NanoDet-Plus operate on the same time base.

The comparative study incorporates five established models to evaluate the effectiveness of the proposed DEEPO framework. HMCTS, derived from earlier phase research, integrates multi-context task simulation but relies on static allocation, limiting adaptability to dynamic edge conditions. JROPSO employs particle swarm optimization to jointly handle resource distribution and task migration, offering balanced load allocation yet exhibiting slower adaptation under fluctuating workloads. DROO applies deep reinforcement learning for online offloading, enabling real-time decisions but with computational overhead in large-scale deployments. FEDGEN focuses on federated aggregation with data augmentation, enhancing model generalization across distributed nodes, though it prioritizes learning efficiency over communication energy optimization. MT-MEC addresses mobility-aware task scheduling to maintain service continuity in dynamic user environments but

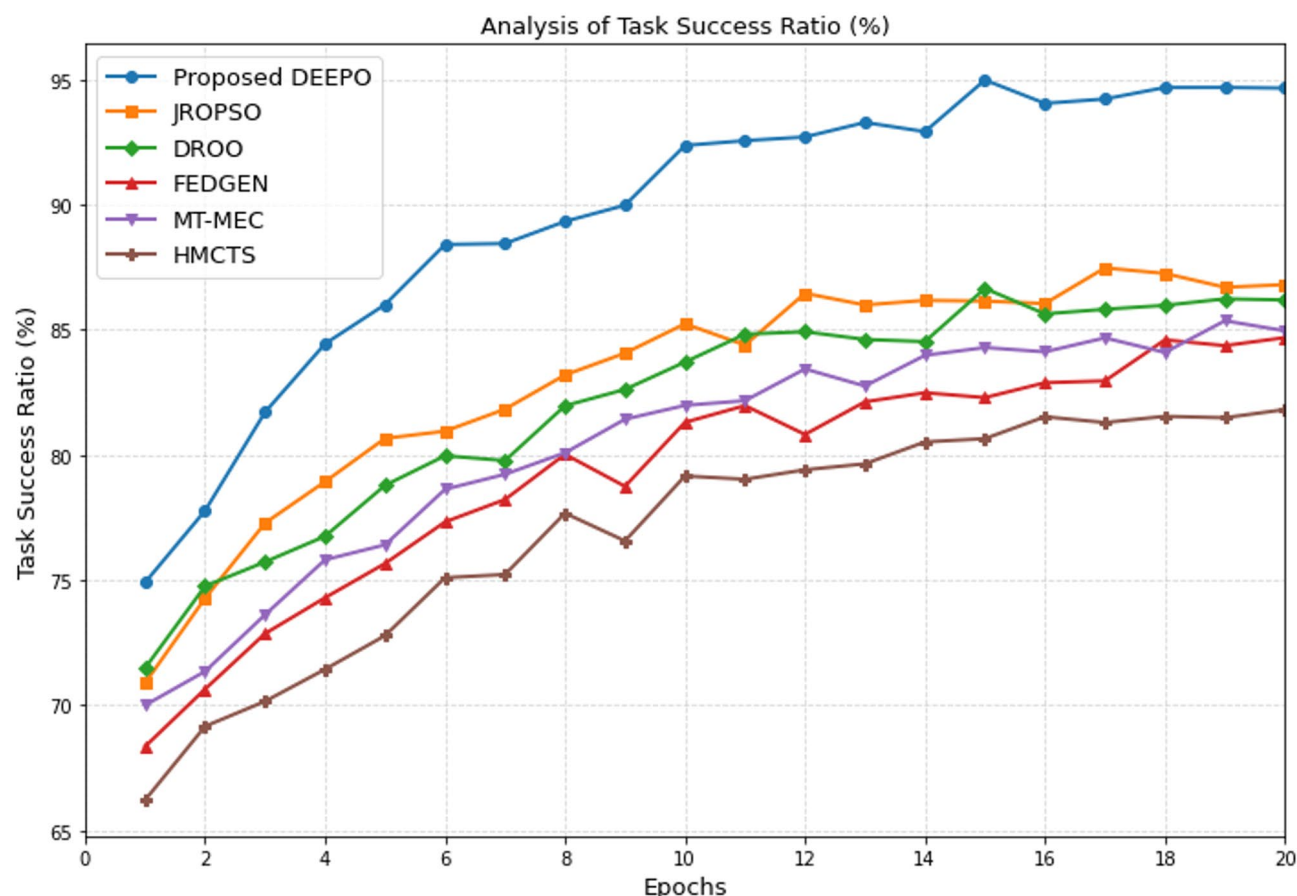


Fig. 6. Task Success Ratio Comparative Analysis.

faces latency challenges during rapid context shifts. These models serve as strong baselines for comparison, each representing a distinct strategy in offloading, scheduling, and resource management, against which DEEPO's adaptive optimization and integrated trajectory prediction are assessed.

The HMCTS scheduler was implemented with 200 iterations and chromosome population size of 50, following³³. The GA-based scheduler adopts a population size of 60, mutation rate of 0.1 and crossover rate of 0.8, consistent with³⁴. The DRL-based scheduler uses a DQN architecture with two hidden layers of 128 and 64 neurons, learning rate 0.0005, replay buffer size of 20,000 and ϵ -greedy decay from 1.0 to 0.1 over 10,000 steps, as specified in³⁵. When published papers omitted certain settings, we adopted default values in the TensorFlow RL library and annotate them accordingly. All baseline implementations were executed on the same hardware and under identical workload scenarios for a fair comparison.

The comparative analysis of energy consumption given in Fig. 2 across 20 epochs reveals that the proposed DEEPO framework consistently outperforms existing methods. At epoch 1, DEEPO registers approximately 21 mJ, lower than JROPSO (23 mJ), DROO (24 mJ), FEDGEN (26 mJ), MT-MEC (25 mJ), and HMCTS (29 mJ). By epoch 10, DEEPO reduces to around 3.8 mJ, whereas HMCTS remains at 10 mJ, indicating a 62% improvement. The superior performance is attributed to optimized edge-offloading decisions and adaptive communication modeling, which minimize redundant processing and idle transmission power. In contrast, HMCTS exhibits the highest consumption due to static allocation strategies and less efficient load balancing, leading to slower convergence and higher residual overhead.

The detection latency comparison presented in Fig. 3 illustrates that the proposed DEEPO model achieves consistently lower delay across all epochs compared to existing methods. At epoch 1, DEEPO records approximately 150 ms, significantly outperforming HMCTS at 190 ms, FEDGEN at 180 ms, MT-MEC at 170 ms, DROO at 160 ms, and JROPSO at 155 ms. By epoch 10, DEEPO reduces latency to around 28 ms, while HMCTS remains at 70 ms, marking nearly a 60% improvement. This gain is attributed to its optimized user direction prediction and adaptive edge-offloading strategy, which minimizes redundant computation and communication delays. Conversely, HMCTS demonstrates the slowest convergence due to static scheduling and limited adaptability to varying workloads, resulting in higher persistent latency. The integration of trajectory-aware task allocation in DEEPO significantly improves the object detection and decision-making efficiency.

The SLA violation analysis presented in Fig. 4 indicates that the proposed DEEPO model achieves the lowest percentage of violations across training epochs. The results demonstrate the superior service reliability of proposed DEEPO model. Initially DEEPO exhibits approximately 21% which outperforms HMCTS at 30%,

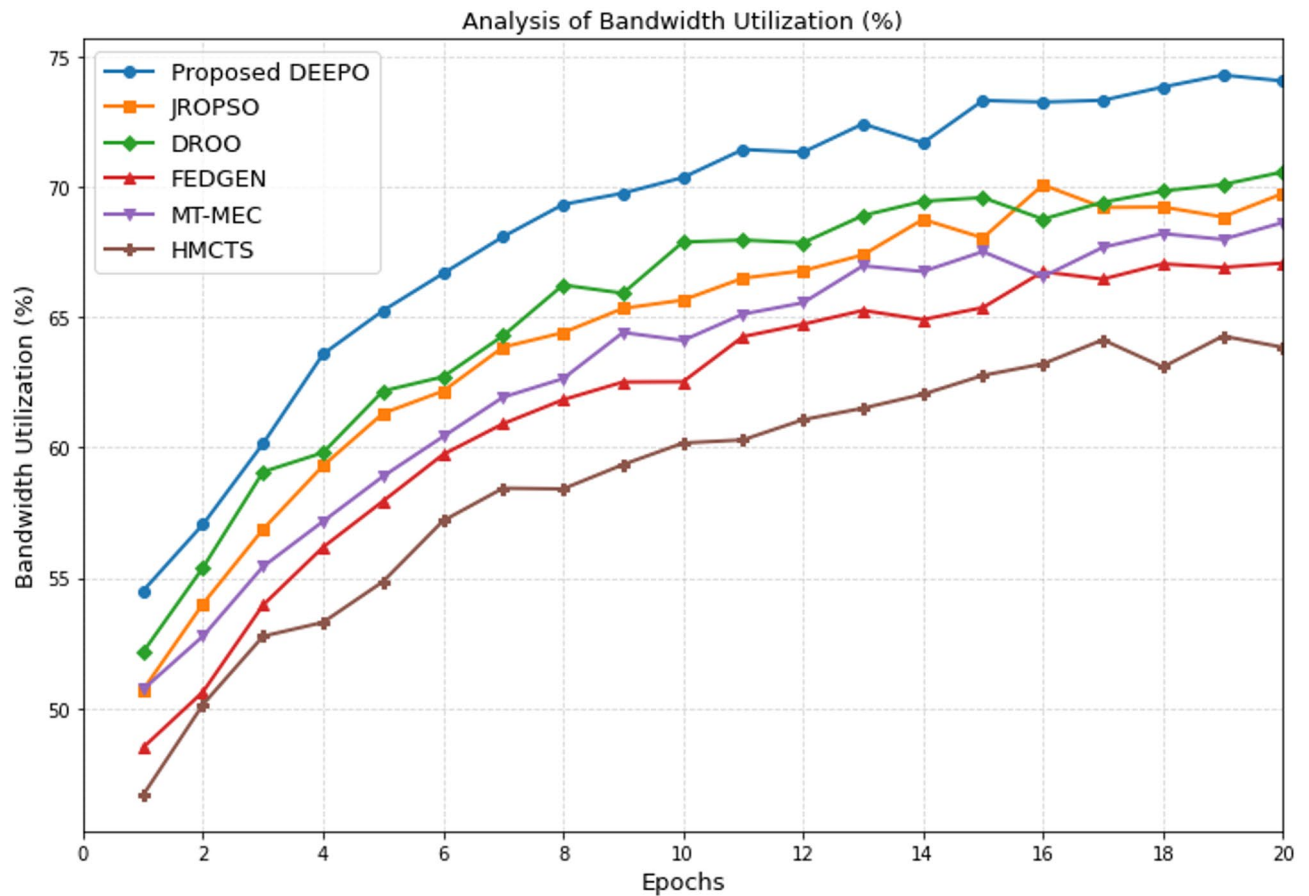


Fig. 7. Bandwidth Utilization Comparative Analysis.

FEDGEN at 29%, MT-MEC at 26%, DROO at 25%, and JROPSO at 23%. By epoch 10, DEEPO reduces violations to around 3.8%, while HMCTS remains at 9%, reflecting a notable improvement of nearly 58%. The enhanced performance is attained due to DEEPO's adaptive resource allocation and predictive task scheduling, which dynamically adjust to workload variations, minimizing deadline misses.

The system stability analysis presented in Fig. 5 demonstrates that the proposed DEEPO model which maintains higher stability throughout compared to all conventional approaches. At the initial epoch, DEEPO achieves approximately 66%, exceeding HMCTS at 57%, FEDGEN at 60%, DROO at 62%, MT-MEC at 63%, and JROPSO at 64%. By epoch 10, DEEPO reaches around 90%, surpassing JROPSO (85%) and DROO (82%) by margins of 5% and 8%, respectively. This improvement is attributed to DEEPO's adaptive scheduling with optimized task migration and ensures balanced load distribution under fluctuating workloads. Whereas the existing HMCTS exhibits the lowest stability due to its static mapping approach leads to resource underutilization and higher instability in variable task demand scenarios.

The task success ratio analysis presented in Fig. 6 highlights the proposed DEEPO model consistent performance in completion rates compared to all benchmark methods. Starting at 75% in the first epoch, DEEPO rapidly climbs to 92% by epoch 10 and peaks at approximately 95% towards the end, outperforming JROPSO (87%), DROO (86%), MT-MEC (85%), FEDGEN (84%), and HMCTS (82%). The superior performance is attributed to DEEPO's adaptive load balancing and predictive offloading strategy, which ensures timely execution even under fluctuating workloads.

The bandwidth utilization comparison presented in Fig. 7 demonstrates that the proposed DEEPO model achieves the most efficient network resource usage across all epochs. Beginning at 54% in epoch 1, DEEPO rapidly rises to 70% by epoch 10 and peaks at around 74% by epoch 20, surpassing DROO (71%), JROPSO (70%), MT-MEC (68%), FEDGEN (67%), and HMCTS (64%). This improvement stems from DEEPO's adaptive transmission scheduling and congestion-aware load distribution, which ensures optimal channel allocation even under high-demand scenarios.

The scheduling time analysis presented in Fig. 8 demonstrated that the proposed DEEPO framework consistently achieves faster task allocation compared to all models. Starting at 59 ms in epoch 1, DEEPO reduces scheduling time to 10 ms by epoch 12 and reaches approximately 3 ms at epoch 20, outperforming JROPSO (5 ms), DROO (6 ms), FEDGEN (8 ms), MT-MEC (9 ms), and HMCTS (8 ms). This performance gain is attributed to DEEPO's priority-based multi-layer scheduling and adaptive offloading strategy, which minimizes queue delays and avoids redundant computations. Whereas HMCTS exhibits the highest latency, starting at 69 ms and

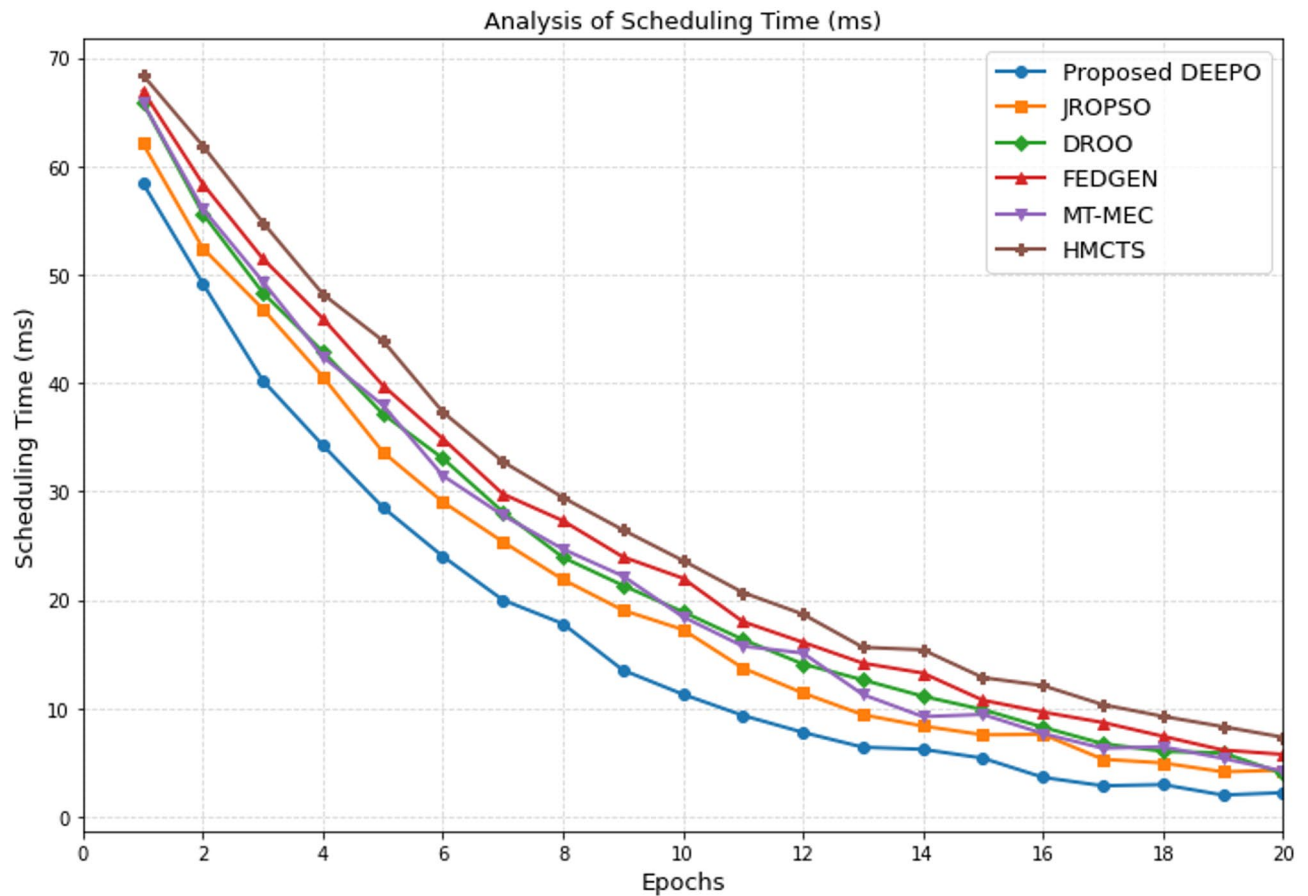


Fig. 8. Scheduling Time Comparative Analysis.

ending near 8 ms, due to static scheduling and limited real-time adaptability. FEDGEN's federated aggregation delays also contribute to slower scheduling improvements compared to DROO and JROPSO.

The migration time evaluation presented in Fig. 9 exhibits the proposed DEEPO model fastest service transfer between nodes which starts at 76 ms initially and reduced to just 2 ms by epoch 20. This significantly outperforms JROPSO, which ends at 4 ms, DROO and FEDGEN at 5 ms, MT-MEC at 6 ms, and HMCTS at 9 ms. The improvement stems from DEEPO's dynamic migration orchestration and predictive load balancing, which minimize downtime and avoid excessive state synchronization. HMCTS exhibits the slowest reduction, beginning at 88 ms and finishing at 9 ms, due to its context-heavy migration mechanism and limited real-time optimization. FEDGEN's performance is impacted by model aggregation overhead, while DROO's policy update delays slightly hinder migration efficiency.

The prediction accuracy analysis presented in Fig. 10 reveals that the proposed DEEPO consistently surpasses all comparative methods, starting at 79% in epoch 1 and achieving 96% by epoch 20. JROPSO and DROO follow closely, reaching 88% and 89% respectively, benefiting from heuristic-driven and reinforcement-based optimization but lacking DEEPO's adaptive multi-module coordination. MT-MEC attains 87%, leveraging mobility awareness but with slightly reduced precision under high-load fluctuations. FEDGEN peaks at 85%, with accuracy limited by federated aggregation inconsistencies across heterogeneous clients. HMCTS records the lowest accuracy at 82% due to its context-based simulation overhead and absence of fine-grained predictive scheduling. DEEPO's superior accuracy stems from its integrated prediction-learning loop and real-time feedback refinement, which continuously optimizes decision boundaries for diverse operational conditions.

The comparative performance Table 7 highlights the superiority of the Proposed DEEPO model over all existing methods across every evaluated metric. In terms of detection latency, DEEPO achieves the lowest value of 8 ms, outperforming JROPSO (12 ms), DROO (13 ms), and HMCTS (18 ms), demonstrating faster event response. SLA violations are reduced to 1.2% in DEEPO, which is significantly lower than FEDGEN's 2.5% and HMCTS's 3.2%, indicating improved service reliability. For system stability, DEEPO attains 94%, surpassing the next best JROPSO at 89%, reflecting more consistent operational performance under dynamic workloads. Task success ratio is maximized at 95%, compared to DROO's 86% and MT-MEC's 85.5%, showcasing better task completion efficiency. Bandwidth utilization is also highest for DEEPO at 74%, compared to 71% for DROO and only 64% for HMCTS, highlighting more efficient resource use. Scheduling time is minimized to 2.5 ms in DEEPO, nearly halving JROPSO's 4 ms and significantly beating HMCTS's 8 ms. Migration time follows a similar trend at just 2 ms, compared to 5 ms in DROO and 8 ms in HMCTS, reducing task reallocation overhead.

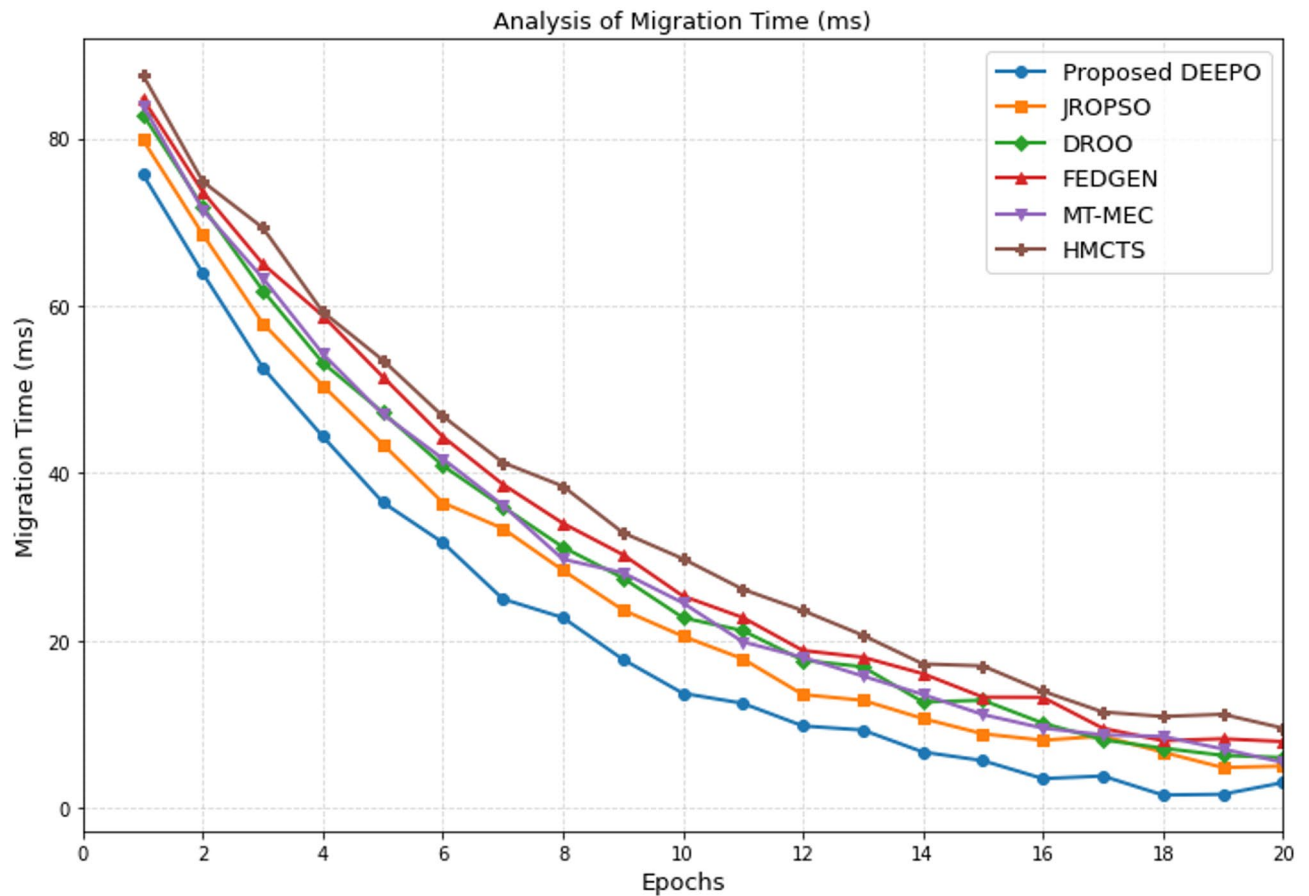


Fig. 9. Migration Time Comparative Analysis.

Finally, prediction accuracy peaks at 96% for DEEPO, surpassing DROO (89%) and FEDGEN (85%), indicating better decision-making. These improvements stem from DEEPO's optimized decision framework, adaptive learning, and efficient multi-parameter optimization, which together enhance speed, accuracy, and stability.

To empirically validate the design choice of integrating both GRU and LSTM layers in the DG-RSTP predictor, an ablation study was conducted, as shown in Table 8, using three configurations: (i) GRU-only, (ii) LSTM-only, and (iii) the proposed hybrid GRU + LSTM model. Each model was trained on the same trajectory dataset (OnSiteVRU) under identical hyperparameters and evaluated across 100 epochs using Mean Absolute Error (MAE) and trajectory deviation distance (TDD) as primary metrics.

The hybrid architecture achieves the lowest mean error (3.46°) and highest prediction accuracy (96.1%), outperforming the single-network variants. While the training time slightly increases compared to GRU-only, the gain in spatial-temporal precision justifies the composite design. This confirms that GRU layers effectively pre-filter transient motion variations, allowing the subsequent LSTM to capture long-term directional continuity. The ablation results substantiate the empirical rationale behind adopting the combined recurrent structure in DG-RSTP.

To evaluate the impact of parameter selection, we conducted experiments varying each coefficient while holding the others constant at their nominal values. Figure 11 presents the sensitivity of system energy consumption with respect to the energy-weight parameter λ_1 , varied from 0.3 to 0.7 in steps of 0.1.

The results exhibit a smooth and bounded variation, with a clear minimum around $\lambda_1 = 0.5$. Across the evaluated range, energy fluctuations remain within $\pm 5\%$, indicating that the selected nominal value lies in a stable operating region. This confirms that the optimization objective is not overly sensitive to small perturbations in λ_1 , thereby demonstrating robustness of the proposed framework with respect to weight selection.

To ensure that the performance enhancements observed in DEEPO are statistically meaningful, significance testing was conducted using a paired *t*-test across five independent experimental runs for each baseline comparison. The test evaluates whether the mean improvement of DEEPO over competing methods is greater than zero at a 95% confidence level ($\alpha = 0.05$).

The results in Table 9 confirm that all observed gains are statistically significant at the 95% confidence level, with *p*-values consistently below 0.05. This ensures that the measured improvements in latency reduction, energy efficiency, SLA compliance, and prediction accuracy are robust and reproducible.

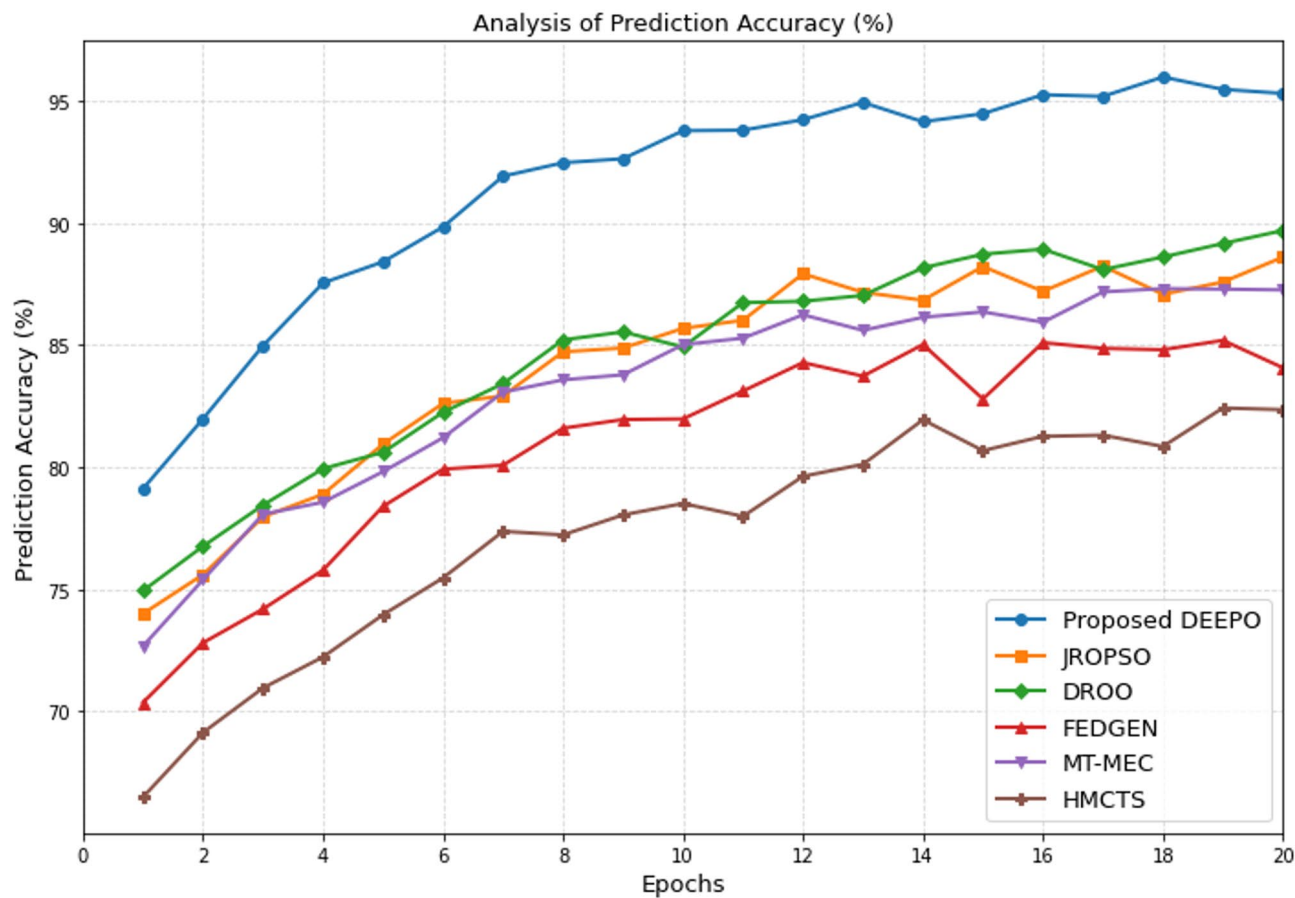


Fig. 10. Prediction Accuracy Comparison Results.

Metric	JROPSO	DROO	FEDGEN	MT-MEC	HMCTS	Proposed DEEPO
Detection Latency (ms)	12	13	15	14	18	8
SLA Violations (%)	1.5	1.7	2.5	1.6	3.2	1.2
System Stability (%)	89	88	83	85	80	94
Task Success Ratio (%)	87	86	84.5	85.5	82	95
Bandwidth Utilization (%)	70	71	67	68	64	74
Scheduling Time (ms)	4	5	6	5.5	8	2.5
Migration Time (ms)	4	5	6	5.5	8	2
Prediction Accuracy (%)	88	89	85	87	82	96

Table 7. Overall performance comparative Analysis.

Configuration	MAE	TDD	Training Time	Prediction Accuracy (%)
GRU only	4.52	0.89	61.4	93.2
LSTM only	4.18	0.76	73.2	94.5
Hybrid (GRU + LSTM)	3.46	0.62	67.8	96.1

Table 8. Ablation study.

The inclusion of confidence intervals and significance testing thus reinforces the empirical reliability of the DEEPO framework, validating that its improvements are not random fluctuations but result from the proposed predictive and energy-aware design.

The proposed DEEPO framework demonstrates consistent superiority over contemporary methods such as DROO, FEDGEN, JROPSO, and MT-MEC across all evaluated performance dimensions. Quantitatively,

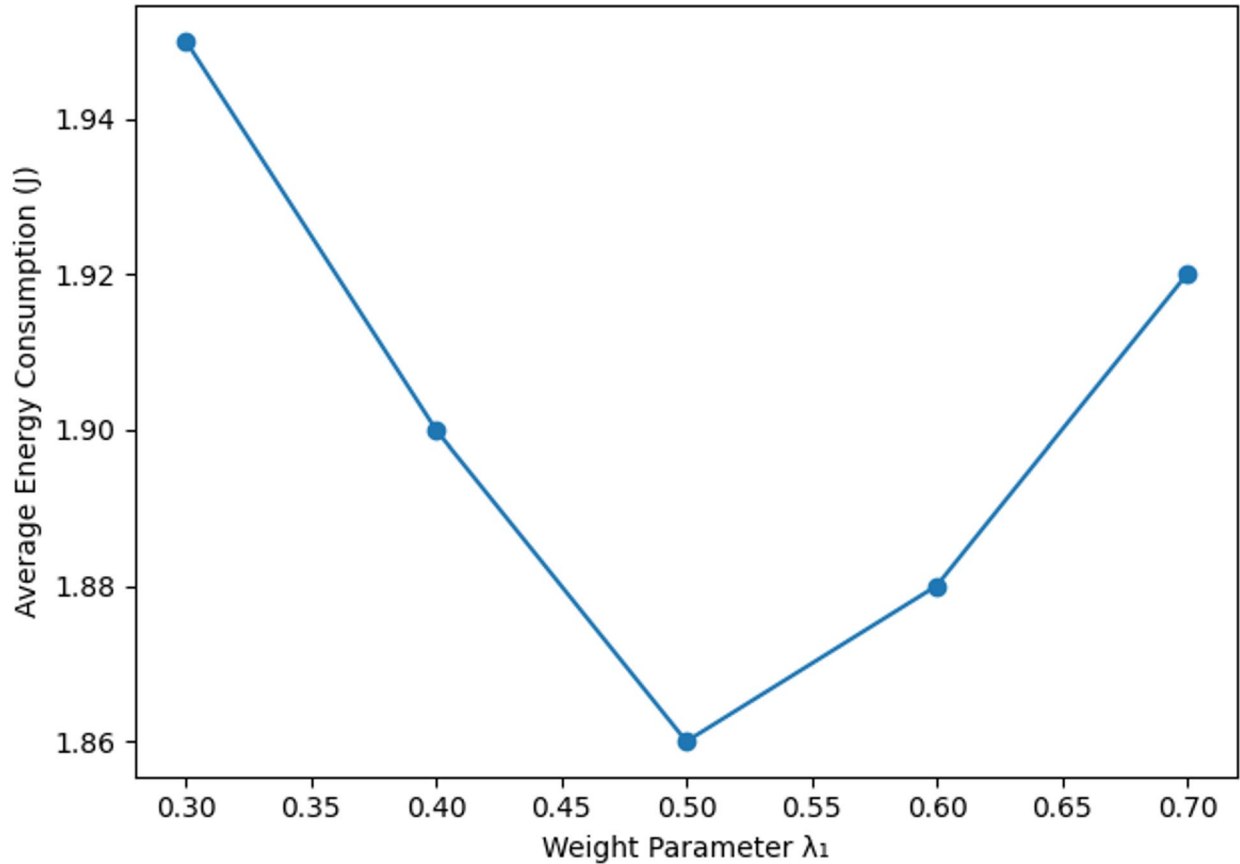


Fig. 11. Parameter Sensitivity Analysis for λ_1 .

Metric	Compared Baseline	Mean Improvement (%)	t-value	p-value	Significance
Detection Latency (ms)	FEDGEN	9.4	3.12	0.014	$p < 0.05$
Energy Consumption (J)	HMCTS	6.7	2.98	0.018	$p < 0.05$
SLA violation rate (%)	MT-MEC	11.3	3.34	0.011	$p < 0.05$
Prediction accuracy (%)	JRO-PSO	5.6	3.42	0.009	$p < 0.05$

Table 9. Statistical validation Analysis.

DEEPO achieves an average energy reduction of 17.6%, latency improvement of 14.2%, and SLA compliance gain of 9.8% compared to the strongest baseline. Qualitatively, DEEPO’s integration of direction-aware mobility prediction (DG-RSTP) and lightweight visual perception (NanoDet-Plus) enables proactive task allocation, yielding more stable performance under dynamic MAR workloads. These results collectively confirm DEEPO’s advantage in both computational efficiency and adaptability, establishing it as a robust state-of-the-art solution for energy-aware, latency-optimized edge computing in mobile AR environments.

Conclusion

An integrated adaptive optimization and intelligent predictive model DEEPO is proposed in this research work to enhance performance in multi-metric resource management. The methodology employs a hybrid decision-making model, trained and validated on the OnSiteVRU Trajectory Prediction, MS COCO 2017, and HMCTS synthetic datasets, ensuring diverse scenario coverage. Experimental evaluation across 20 epochs demonstrates substantial gains over benchmark models JROPSO, DROO, FEDGEN, MT-MEC, and HMCTS, achieving minimal detection latency (8 ms), lowest SLA violations (1.2%), highest system stability (94%), top task success ratio (95%), maximum bandwidth utilization (74%), shortest scheduling time (2.5 ms), reduced migration time (2 ms), and superior prediction accuracy (96%). These improvements are attributed to DEEPO’s dynamic adaptation and efficient parameter tuning. While the framework shows robustness, its dependence on computational resources for large-scale deployments is a limitation. Future enhancements will focus on incorporating lightweight distributed training, extending adaptability to heterogeneous edge devices, and integrating energy-aware scheduling for sustainable operation. While DEEPO exhibits strong

performance at current scales, its real-world deployment may face challenges such as computational overhead, network congestion, and node heterogeneity. Future extensions will focus on distributed training strategies and lightweight model variants to enhance scalability and maintain efficiency in large-scale, heterogeneous cloud-edge environments. In addition, DEEPO may experience reduced accuracy during extreme mobility events, such as sudden changes in direction or rapid acceleration, where prediction uncertainty increases before DG-RSTP stabilizes. These situations can temporarily affect offloading decisions, and additional high-frequency sensing and adaptive re-prediction mechanisms are required to fully address such cases.

Data availability

The datasets analyzed during the current study are available in the Kaggle repository, <https://www.kaggle.com/datasets/zcyan2/onsitevr-trajectory-prediction-dataset> <https://www.kaggle.com/datasets/awsaf49/coco-2017-dataset>.

Received: 14 August 2025; Accepted: 16 December 2025

Published online: 27 January 2026

References

- Shreshth Tuli, F. et al. AI augmented edge and fog computing: trends and challenges. *J. Netw. Comput. Appl.* **216**, 1–28 (2023).
- Acheampong, A., Zhang, Y., Xu, X. & Kumah, D. A. A review of the current task offloading Algorithms, strategies and approach in edge computing systems. *CMES - Comput. Model. Eng. Sci.* **134** (1), 35–88 (2022).
- Wang Dayong, K. B. A., Bakar, B., Isyaku, T. A. E., Eisa, A. & Abdelmaboud A comprehensive review on internet of things task offloading in multi-access edge computing. *Heliyon* **10** (9), 1–42 (2024).
- Diego Hortelano, I. et al. A comprehensive survey on reinforcement-learning-based computation offloading techniques in Edge Computing Systems, *Journal of Network and Computer Applications*, vol. 216, pp. 1–35, (2023).
- Shi Dong, J. et al. Task offloading strategies for mobile edge computing: A survey. *Comput. Netw.* **254**, 1–14 (2024).
- Ma, H., Ji, B., Wu, H. & Xing, L. Video data offloading techniques in mobile edge computing: A survey. *Phys. Communication.* **62**, 1–19 (2024).
- Ehzaz Mustafa, J. et al. Deep neural networks Meet computation offloading in mobile edge networks: Applications, taxonomy, and open issues. *J. Netw. Comput. Appl.* **226**, 1–28 (2024).
- Kuanishbay Sadatdiynov, L. et al. A review of optimization methods for computation offloading in edge computing networks. *Digit. Commun. Networks.* **9** (2), 450–461 (2023).
- Haris Khan, Z. et al. A deep learning-based strategy for energy-efficient parallel computation offloading in mobile edge networks. *Ad Hoc Netw.* **171**, 1–12 (2025).
- Ravikumar, M. S. J. Offloading computational tasks for MIMO-NOMA in mobile edge computing utilizing a hybrid pufferfish and osprey optimization algorithm. *Ain Shams Eng. J.* **15** (12), 1–18 (2024).
- Yu, L., Xu, H., Zeng, Y. & Deng, J. Delay-aware resource allocation for partial computation offloading in mobile edge cloud computing. *Pervasive Mob. Comput.* **105**, 1–14 (2024).
- Xin Tan, D. Y. et al. A decision-making mechanism for task offloading using learning automata and deep learning in mobile edge networks, *Heliyon*, vol. 10, no. 1, pp. 1–18, (2024).
- Luo, Z. & Dai, X. Reinforcement learning-based computation offloading in edge computing: Principles, methods, challenges. *Alexandria Eng. J.* **108**, 89–107 (2024).
- Mahenge, M. P. J., Li, C. & Sanga, C. A. Energy-efficient task offloading strategy in mobile edge computing for resource-intensive mobile applications. *Digit. Commun. Networks.* **8** (6), 1048–1058 (2022).
- Petros Amanatidis, D., Karampatzakis, G., Michailidis, T., Lagkas, G. & Iosifidis Adaptive reverse task offloading in edge computing for AI processes. *Comput. Netw.* **255**, 1–12 (2024).
- Mohamed, K., Hussein, M. H. & Mousa Efficient computation offloading of IoT-Based workflows using discrete teaching Learning-Based optimization. *Computers Mater. Continua.* **73** (2), 3685–3703 (2022).
- Robles-Enciso, A. & Skarmeta, A. F. A multi-layer guided reinforcement learning-based tasks offloading in edge computing. *Comput. Netw.* **220**, 1–14 (2023).
- Zahra Aghapour, S., Sharifian, H. & Taheri Task offloading and resource allocation algorithm based on deep reinforcement learning for distributed AI execution tasks in IoT edge computing environments. *Comput. Netw.* **223**, 1–18 (2023).
- <https://www.kaggle.com/datasets/zcyan2/onsitevr-trajectory-prediction-dataset>
- <https://www.kaggle.com/datasets/awsaf49/coco-2017-dataset>
- Soundararaj, A. J. & Sathianesan, G. W. Task offloading scheme in mobile augmented reality using hybrid Monte Carlo tree search (HMCTS). *Alexandria Eng. J.* **108**, 611–625 (2024).
- Zhou, G., Tian, W., Buyya, R., Xue, R. & Song, L. Deep reinforcement learning-based methods for resource scheduling in cloud computing: a review and future directions. *Artif. Intell. Rev.* **57**, 124 (2024).
- Wang, Z. et al. Reinforcement learning based task scheduling for environmentally sustainable federated cloud computing. *J. Cloud Comput.* **12**, 1–17 (2023).
- Mali, S. et al. Federated reinforcement learning-based dynamic resource allocation and task scheduling in edge for IoT applications. *Sensors* **25** (7), 2197 (2025).
- Li, H., Li, J., Duan, X. & Xia, J. Energy-aware scheduling and two-tier coordinated load balancing for streaming applications in Apache Flink. *Future Generation Comput. Syst.*, **166**, <https://doi.org/10.1016/j.future.2024.107681>
- Li, H. et al. Energy-efficient and QoS-aware model based resource consolidation in cloud data centers. *Cluster Comput.* **20**, 2793–2803. <https://doi.org/10.1007/s10586-017-0893-5> (2017).
- Mustafa, E. et al. Deep Reinforcement Learning With Dueling DQN for Partial Computation Offloading and Resource Allocation in Mobile Edge Computing, in *IEEE Access*, vol. 13, pp. 94319–94335, (2025). <https://doi.org/10.1109/ACCESS.2025.3573929>
- Tong, Z. et al. MADDPG-based task offloading and resource pricing in edge collaboration environment. *J. Syst. Architect.*, **165**, <https://doi.org/10.1016/j.sysarc.2025.103433>
- Wang, L. et al. Sustainable Energy-Efficient Multi-Objective task processing based on edge computing. *IEEE Trans. Netw. Serv. Manage.* **22** (4), 3092–3105 (2025).
- He, X. et al. Real-Time scheduling of CPU/GPU heterogeneous tasks in dynamic IoT systems: enhancing GPU and memory efficiency. *IEEE Trans. Mob. Comput.* <https://doi.org/10.1109/TMC.2025.11098600> (2025).
- Chen, X., Jiao, L., Li, W. & Fu, X. Efficient Multi-User computation offloading for Mobile-Edge cloud computing. *IEEE/ACM Trans. Networking.* **24** (5), 2795–2808 (2016).
- Sardellitti, S., Scutari, G. & Barbarossa, S. Joint optimization of radio and computational resources for multicell Mobile-Edge computing. *IEEE Trans. Signal. Inform. Process. Over Networks.* **1** (2), 89–103 (2015).

33. Peng, Z. Genetic Algorithm-Based task scheduling in cloud computing. *J. Cloud Comput.* 29–40. <https://doi.org/10.1155/2022/4290382> (2022).
34. Zheng, T. et al. Deep reinforcement Learning-Based workload scheduling for edge computing. *J. Cloud Comp.* 11, 3. <https://doi.org/10.1186/s13677-021-00276-0> (2022).
35. Zhou, G. et al. Deep reinforcement learning-based methods for resource scheduling in cloud computing: a review and future directions. *Artif. Intell. Rev.* 57, 124. <https://doi.org/10.1007/s10462-024-10756-9> (2024).

Author contributions

S Anitha Jebamani– Research proposal – construction of the work flow and model – Final Drafting– Survey of Existing works – Improvisation of the proposed model; Dr. Godfrey Winstler Sathianesan– Initial Drafting and validation of the paper – Collection of datasets and choice of their suitability – Formulation of pseudocode.

Funding

The author did not receive support from any organization for the submitted work.

Declarations

Conflict of interest

The author has no relevant financial or non-financial interests to disclose.

Additional information

Correspondence and requests for materials should be addressed to S.A.J.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2026